Machine Learning with Applications 12 (2023) 100470

Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa

Deep learning for anomaly detection in log data: A survey

Max Landauer [a,*], Sebastian Onder [b], Florian Skopik [a], Markus Wurzenberger [a]

[a] Austrian Institute of Technology, Giefinggasse 4, 1210 Vienna, Austria
[b] Vienna University of Technology, Karlsplatz 13, 1040 Vienna, Austria

ARTICLE INFO

Keywords:
Log data
Anomaly detection
Neural networks
Deep learning

ABSTRACT

Automatic log file analysis enables early detection of relevant incidents such as system failures. In particular, self-learning anomaly detection techniques capture patterns in log data and subsequently report unexpected log event occurrences to system operators without the need to provide or manually model anomalous scenarios in advance. Recently, an increasing number of approaches leveraging deep learning neural networks for this purpose have been presented. These approaches have demonstrated superior detection performance in comparison to conventional machine learning techniques and simultaneously resolve issues with unstable data formats. However, there exist many different architectures for deep learning and it is non-trivial to encode raw and unstructured log data to be analyzed by neural networks. We therefore carry out a systematic literature review that provides an overview of deployed models, data pre-processing mechanisms, anomaly detection techniques, and evaluations. The survey does not quantitatively compare existing approaches but instead aims to help readers understand relevant aspects of different model architectures and emphasizes open issues for future work.

Contents

1. Introduction 2
2. Background 3
2.1. Preliminary definitions 3
2.1.1. Deep learning 3
2.1.2. Log data 3
2.1.3. Anomaly detection 3
2.2. Challenges 3
3. Survey method 4
3.1. Search strategy 4
3.1.1. Initial literature collection with search string 4
3.1.2. Selection of relevant publications 4
3.2. Reviewed features 5
4. Survey results 5
4.1. Bibliometrics 5
4.1.1. Publications per year 5
4.1.2. Citations 6
4.2. Deep learning techniques 6
4.2.1. Deep learning models 6
4.2.2. Training loss function 8
4.2.3. Operation mode 8
4.3. Log data preparation 8
4.3.1. Pre-processing 9
4.3.2. Event grouping 9
4.3.3. Feature extraction 10
4.3.4. Feature representation 10

* Corresponding author.
E-mail addresses: max.landauer@ait.ac.at (M. Landauer), e1621235@student.tuwien.ac.at (S. Onder), florian.skopik@ait.ac.at (F. Skopik), markus.wurzenberger@ait.ac.at (M. Wurzenberger).

https://doi.org/10.1016/j.mlwa.2023.100470
Received 23 March 2023; Received in revised form 18 April 2023; Accepted 23 April 2023
Available online 12 May 2023
2666-8270/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

4.4. Anomaly detection techniques 11
4.4.1. Anomaly types 11
4.4.2. Network output 11
4.4.3. Detection method 11
4.5. Evaluation & reproducibility 12
4.5.1. Data sets 12
4.5.2. Evaluation metrics 14
4.5.3. Benchmark approaches 14
4.5.4. Reproducibility 15
5. Discussion 15
6. Conclusion 17
CRediT authorship contribution statement 17
Declaration of competing interest 17
Data availability 17
Acknowledgments 17
References 17

1. Introduction

Log files provide a rich source of information when it comes to monitoring computer systems. Thereby, the majority of log events are usually generated as consequences of normal system operations, such as starting and stopping of processes, restarting of virtual machines, users accessing resources, etc. However, applications also produce logs when faulty or otherwise undesired system states occur, for example, failed processes, availability issues, or security incidents. These traces of unexpected and possibly unsafe system activities are important for system operators that timely need to act upon them to prevent or diminish system damage and avoid adverse cascading effects.

The main problem for this kind of log file analysis is that it is non-trivial to identify these relevant log events within the much larger number of less interesting traces of standard system usage. In particular, the sheer amount of logs produced by modern applications renders manual analysis infeasible and necessitates automatic mechanisms (Wang, Zhang, Wang & Cao, 2021). Unfortunately, manually coded signatures and rules that search for specific keywords in logs only have limited applicability and are not suitable for scenarios that are not known beforehand (Liao, Lin, Lin, & Tung, 2013). It is therefore necessary to deploy anomaly detection techniques that automatically learn models representing the normal baseline of system behavior and subsequently disclose any deviations from these models as possibly adverse activities that require attention by human operators.

Machine learning provides many viable techniques for the purpose of anomaly detection in log files and many different approaches have been proposed in the past, including clustering (Landauer, Skopik, Wurzenberger, & Rauber, 2020) and workflow mining (He, Zhu, He, & Lyu, 2016), statistical analysis of event parameters (Kruegel & Vigna, 2003), time-series analysis to recognize changes of event frequencies (Landauer, Wurzenberger, Skopik, Settanni, & Filzmoser, 2018), and many more (Chandola, Banerjee, & Kumar, 2009; Liao et al., 2013). Recently, researchers started using deep neural networks for log-based anomaly detection in an attempt to repeat the successes of deep learning from image and speech recognition that outperform conventional machine learning methods (LeCun, Bengio, & Hinton, 2015). However, as system log events are generally unstructured and involve intricate dependencies, it is non-trivial to prepare the data in a way to enable ingestion by neural networks and extract features that are relevant for detection. Moreover, the wide variety of existing deep learning architectures such as recurrent or convolutional neural networks makes it difficult to select an appropriate model for a specific use-case at hand and understand their respective requirements on the format and properties of the input data.

To the best of our knowledge there is currently only a limited overview of the state-of-the-art of log-based anomaly detection with deep learning (Chen, Liu, Gu, Su, & Lyu, 2021; Kwon et al., 2019; Le & Zhang, 2022; Yadav, Kumar, & Dhavale, 2020). As a consequence it is difficult to understand what features are suitable to be extracted from raw log data, how these features could be transformed into a format that is adequate to be ingested by neural networks, and which model architectures are appropriate for detecting specific patterns in logs. Existing surveys only compare few anomaly detection approaches and focus mainly on sequential patterns in log data (Chen, Liu, et al., 2021; Le & Zhang, 2022; Yadav et al., 2020), present broad studies on system log data analysis that do not sufficiently cover deep learning models and challenges (Bhanage, Pawar, & Kotecha, 2021; Zhao, Wang, et al., 2021), or focus on network traffic rather than system log data (Kwon et al., 2019).

We therefore carry out a systematic literature review on deep learning for anomaly detection in log data. Our main focus is thereby to survey scientific publications on the deployed model architectures, their respective requirements and transformations for handling unstructured input log data, the methods used to differentiate between normal and anomalous data samples, and the presented evaluations. The results of this study are beneficial for researchers and industries alike, because a better understanding of challenges and features of different deep learning algorithms avoids pitfalls when developing anomaly detection techniques and eases selection of existing detection systems for both academic and real-world use-cases. Moreover, a detailed investigation of pre-processing strategies is essential to utilize all information available in the logs when carrying out anomaly detection and to understand the influence of data representations on the detection capabilities, in particular, what types of anomalies can be detected under which circumstances. Regarding scientific evaluations, we particularly pay attention to relevant aspects of experiment design, including data sets, metrics, and reproducibility, to point out deficiencies in prevalent evaluation strategies and suggest remedies. Finally, our study also aims to create a work of reference and establish a starting point for future research. We point out that this survey does not quantitatively compare detection performances of the reviewed approaches as only few open-source implementations are available and comparisons of these approaches are already presented in other surveys (Chen, Liu, et al., 2021; Le & Zhang, 2022). In alignment with the aforementioned goals we formulate the research questions of this survey as follows:

RQ1: What are the main challenges of log-based anomaly detection with deep learning?
RQ2: What state-of-the-art deep learning algorithms are typically applied?
RQ3: How is log data pre-processed to be ingested by deep learning models?
RQ4: What types of anomalies are detected and how are they identified as such?
RQ5: How are the proposed models evaluated?
RQ6: To what extent do the approaches rely on labeled data and support incremental learning?

RQ7: To what extent are the presented results reproducible in terms of availability of source code and used data?

The remainder of this paper is structured as follows. Section 2 first explains the terms deep learning, log data, and anomaly detection, and then provides an overview of common challenges. We explain our methodology for selecting relevant publications and carrying out the survey in Section 3. Section 4 presents all results of our survey in detail. We discuss these results and answer our research questions in Section 5. Finally, Section 6 concludes this paper.

2. Background

In this section we first clarify some general concepts and terms relevant for anomaly detection in log data based on deep learning. We then outline scientific challenges that are specific to that research field.

2.1. Preliminary definitions

The study carried out in this paper hinges on an understanding of three main concepts: deep learning, log data, and anomaly detection. However, the exact characteristics and consequential requirements of the respective fields may be used differently across research areas and existing literature. In the following, we therefore describe the basic properties of these three concepts.

2.1.1. Deep learning

Artificial neural networks (ANN) have been developed in an attempt to recreate biological information processing systems in the form of connected communication nodes. For this purpose, varying numbers of nodes are arranged in sequences of layers, in particular, an input layer that reads in the data, several hidden layers connecting neighboring layers with weighted edges, and an output layer. Nodes activate when receiving specific signals on their connected edges, which in turn generates the input for subsequent layers. The main idea is that such networks are capable of recognizing non-linear structures in the input data and subsequently classifying the processed instances through training, which involves minimizing the error of classifications by adjusting the weights of edges accordingly. Thereby, ANN enable supervised training where labels for all classes are available (i.e., data samples are marked with labels such as normal or anomalous), semi-supervised training where labels of some classes are available, as well as unsupervised learning where no labels are available.

In general, deep learning algorithms are understood as neural networks with multiple hidden layers. Several different architectures of deep neural networks have been proposed in the past, such as recurrent neural networks (RNN) for sequential input data. Deep learning has been shown to outperform conventional machine learning methods (e.g., support vector machines or decision trees) and even human experts in many application areas such as image classification, speech recognition, and many more (LeCun et al., 2015; Sarker, 2021b).

2.1.2. Log data

Log data are a chronological sequence of single- or multi-line events generated by applications to permanently capture specific system states, in particular, for manual forensic analysis in case that failures or other unexpected incidents occur. Log events are usually available in textual form and range from structured vectors (e.g., comma-separated values) over semi-structured objects (e.g., key–value pairs) to unstructured human-readable messages with heterogeneous event types. Despite the fact that no unified log format exists, log events usually contain their generation time stamp as one of their event parameters. Other parameters that are sometimes present in different types of log data are logging levels (e.g., *INFO* or *ERROR*) or process identifiers that link sequences of related events (Landauer, Skopik, Wurzenberger, & Rauber, 2020).

While single log events describe (part of) the system state in one particular point in time, groups of log events represent the dynamic workflows of the underlying program logic. The reason for this is that log events are generated by print statements purposefully placed by software developers throughout their code to support understanding of program activities and debugging. These statements comprise of static parts, i.e., hard-coded strings, and variable parts, i.e., parameters that are dynamically determined during program runtime. In the past, a large amount of research was directed towards automatic extraction of so-called log keys (also known as log signatures, log templates, or simply log events) that represent templates for the original print statements and enable parsing of logs (Zhu et al., 2019). These parsers allow to derive values from logs that are more suitable to be used for subsequent analysis than unstructured log messages, in particular, through (i) assignment of event type identifiers to log events and (ii) extraction of parameters from log messages (Bao et al., 2018).

2.1.3. Anomaly detection

Anomalies are those instances in a data set that exhibit rare or otherwise unexpected characteristics and thus stand out from the rest of the data (Chandola et al., 2009). For the purpose of detection, the conformity of these data instances is usually measured through one or more continuous or categorical attributes that are associated with each instance and enable the computation of similarity metrics. For independent data, it is sufficient to declare single or small groups of instances with high dissimilarities to all other data points as outliers, which are also referred to as point anomalies. For all data where instances are not independent from each other, e.g., all kinds of ordered data including log data, two additional types of anomalies occur. First, contextual anomalies are instances that are only anomalous with respect to the context they occur (but not otherwise), such as the time of occurrence. For example, consider the start of a daily executed data backup procedure that suddenly takes place outside of the scheduled times. Second, collective anomalies are groups of instances that are only anomalous due to their combined occurrence (but not individually), such as a specific sequence of log events.

An implicit assumption of most anomaly detection techniques is that the analyzed data holds far fewer anomalies than normal instances. This enables that detection takes place in a fully unsupervised manner, i.e., no labeled data is necessary to train the models. However, many scientific approaches instead pursue semi-supervised detection, where training data containing only normal instances are available and evaluation then takes place on a test data set comprising both normal and anomalous instances. The main advantages of semi-supervised operation is that anomalous instances are not learned by the models and that it is often relatively simple to gather normal data, while modeling and labeling anomalies is less straightforward. Accordingly, approaches for supervised anomaly detection have lower applicability and are comparatively rare (He et al., 2016).

2.2. Challenges

Log-based anomaly detection has been an active field of research for decades. Thereby, most of the presented approaches rely on conventional machine learning techniques. However, the last few years have seen a strong increase of approaches that leverage deep learning to disclose anomalous log events that relate to unexpected system behavior. In the following, we summarize the main challenges that need to be overcome for effective and applicable detection.

- **Data representation**. Deep learning systems generally consume structured and numeric input data. It is non-trivial to feed log data into neural networks as they frequently involve a mix of heterogeneous event types, unstructured messages, and categorical parameters (Chalapathy & Chawla, 2019; Yadav et al., 2020).

- **Data instability**. As applications evolve, new log event types may occur that differ from the ones in the training data. In addition, the observed system behavior patterns are subject to change as technological environments and their utilization vary over time. Deep learning systems therefore need to incrementally update their models and adapt their baseline for normal system behavior to enable real-time detection (Chalapathy & Chawla, 2019; Chen, Liu, et al., 2021; Le & Zhang, 2022; Yadav et al., 2020; Zhang et al., 2019).
- **Class imbalance**. Anomaly detection inherently assumes that normal events outnumber anomalous ones. Many approaches based on neural networks are known to perform sub-optimally for imbalanced data sets (Chalapathy & Chawla, 2019; Le & Zhang, 2022).
- **Anomalous artifact diversity**. Manifestations of anomalies affect log events as well as parameters thereof in various ways, including changes of sequential patterns, frequencies, correlations, inter-arrival times, etc. Detection techniques are often designed only for properties of specific anomaly types and are therefore not generally applicable.
- **Label availability**. As anomalies represent unexpected system behavior, there are generally no labeled anomaly instances available for training. This restricts applications to semi- and unsupervised deep learning systems, which are known to achieve lower detection performance than supervised approaches (Chen, Liu, et al., 2021; Le & Zhang, 2022).
- **Stream processing**. Logs are generated as a continuous stream of data. To enable on-the-fly monitoring rather than forensic analysis, deep learning systems need to be designed for single-pass data processing when it comes to detection and model updating (Landauer, Skopik, Wurzenberger, & Rauber, 2020; Le & Zhang, 2022).
- **Data volume**. Log data is generated in high volumes, with some systems producing millions (Xu, Huang, Fox, Patterson, & Jordan, 2009b) or even billions (Mi, Wang, Zhou, Lyu, & Cai, 2013) of events daily. Efficient algorithms are required to ensure real-time processing in practical applications, in particular, when running on machines with few computational resources such as edge devices.
- **Interleaving logs**. Sequences of related log events may be interleaving each other when many processes operate simultaneously or distributed logs are collected centrally. It is non-trivial to retrieve the original event sequences when events lack session identifiers (Le & Zhang, 2022).
- **Data quality**. Low data quality may be the result of improper log collection or technical issues during log generation and cause negative effects on machine learning effectiveness. Common problems involve incorrect time stamp information, event ordering, missing events, duplicated records, mislabeled events, etc. Fischer et al. (2020), Suriadi, Andrews, ter Hofstede, and Wynn (2017).
- **Model explainability**. Approaches based on neural networks generally suffer from a lower explainability than conventional machine learning methods. Difficulties to understand the reasons behind both correct and incorrect classifications are especially problematic when it comes to making justified decisions in response to critical system behavior or security incidents (Chalapathy & Chawla, 2019; Chen, Liu, et al., 2021).

3. Survey method

This section outlines the method that was used to carry out the systematic literature review. We first describe our strategy for collecting relevant literature and then present the evaluation criteria that we used to analyze the retrieved papers.

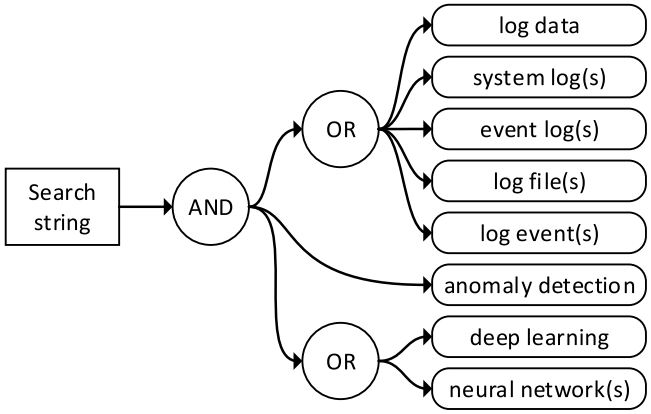


Fig. 1. Composition of the search string used to retrieve relevant literature.

3.1. Search strategy

In this section we describe the process of gathering relevant publications to be included in the survey. First, an initial collection of literature is collected using a web search. Subsequently, relevant papers are selected using inclusion, exclusion, and quality criteria.

3.1.1. Initial literature collection with search string

In order to obtain an initial set of approaches from the state-of-the-art, we assemble a search string to query common databases for scientific publications. In particular, we design the search string so that only publications containing the three main concepts relevant for this survey are retrieved: log data, anomaly detection, and deep learning. Since some publications use different terminology and to decrease the likelihood that relevant publications are missed, we also use the terms "system log(s)", "event log(s)", "log file(s)", "log event(s)" as alternatives for "log data", and "neural network(s)" as an alternative for "deep learning". We omit the term "log" without any other word as it yields many results that contain logarithms but are not relevant for our study. Fig. 1 displays the final search string.

We then use this search string to gather publications from the following databases: Science Direct,[1] Scopus,[2] SpringerLink,[3] ACM Digital Library,[4] IEEE Xplore,[5] Google Scholar,[6] and Web of Science.[7] The search was conducted in January of 2022 and returned a total of 2925 publications. In the following section, we describe our method for selecting relevant publications from this set.

3.1.2. Selection of relevant publications

To sort out publications that are not relevant for this survey and reduce the set of publications to a manageable size, we define multiple selection criteria and apply them on our initial collection. Our main criterion for including the publication in the survey is as follows: *The model proposed in the publication applies deep learning techniques (i.e., a multi-layered neural network) for anomaly detection in heterogeneous and unstructured log data*. Moreover, we define several exclusion criteria that we use to omit publications with low relevance or otherwise inappropriate format. The list of exclusion criteria is as follows.

[1] https://www.sciencedirect.com/
[2] https://www.scopus.com/
[3] https://link.springer.com/
[4] https://dl.acm.org/
[5] https://ieeexplore.ieee.org/
[6] https://scholar.google.com/
[7] https://www.webofscience.com/

- There is no indication stated in the paper that the presented approach is applicable or designed for application with log data. Our survey does not attempt to adopt methods from other domains for log data analysis.
- There exists a more recent publication that presents the same or a similar study. The purpose of this criteria is to ensure that the most up-to-date versions of specific approaches are analyzed.
- The publication only applies an existing approach without novel modifications from the original concept.
- The publication is in any language other than English.
- The publication is not available in electronic form.
- The publication is a book, technical report, lecture note, presentation, review, or thesis. In these cases, the corresponding conference or journal publications were reviewed if possible.

Note that we do not constraint the publications to a specific time range in order to avoid missing any older publications that are nonetheless relevant for this survey. However, we aim to omit publications of generally lower quality that do not meet the minimum scientific standards. We therefore only select publications that meet the following quality criteria in addition to our main inclusion criterion.

- The purpose of the study and its findings are explicitly stated, e.g., the design of a deep learning system for the purpose of anomaly detection is stated as the main contribution of the paper.
- The applied deep learning models and their parameters are rigorously described, i.e., it is clear to the reader what type of deep learning model was selected and how its layout was designed, e.g., how many layers it comprises.
- The publication includes a sound evaluation of the presented approach, comprising a convincing motivation for the design of the conducted experiments, a comprehensive description of the evaluation process and overall setup, an explanation for choosing the captured metrics, and a detailed discussion of the gathered results and their implications.
- The data sets used for evaluating the approach are referenced or described. This criteria ensures that our survey does not include publications presenting potentially misleading findings originating from data sets that are inadequate for anomaly detection.
- Visualizations are clear and readable. Incomprehensible presentation of results are misleading and lack scientific value.

Our selection procedure is a two-stage process. First, we reduce the initial collections of publications using the inclusion and exclusion criteria based on the title and abstract of each paper. After this stage 331 publications remained. In the second stage, we carry out the selection using all aforementioned criteria based on the contents of each paper. We eventually obtained 62 papers that were included in this survey. The following section outlines our method for analyzing these publications.

3.2. Reviewed features

To ensure that we analyze the selected publications on a common scheme and address our research questions, we formulate a list of features that we assess for each paper. The following set of questions concerns the applied deep learning (DL) model and mode of operation.

DL-1: Which deep learning models are used?
DL-2: Which training loss functions are applied?
DL-3: Does the approach support online or incremental[8] learning?
DL-4: Does training take place in un-, semi-, or supervised manner?

We then analyze the different ways how the reviewed approaches feed raw log data into deep learning models. The following questions therefore address the pre-processing (PP) and transformation of logs into numeric vector or matrix representations.

PP-1: How are raw logs pre-processed?
PP-2: What features are extracted from pre-processed logs?
PP-3: How are extracted features represented as vectors?

The next set of questions deals with the anomaly detection (AD). In particular, we are interested in the different types of anomalies to understand whether they are linked to the features extracted from the raw logs and their representations as vectors.

AD-1: What types of anomalies are detected by the approach?
AD-2: How is the output of the deep neural network[9] used for anomaly detection?
AD-3: How are anomalies differentiated from normal data samples?

Utilizing openly accessible data sets for evaluations as well as publishing source code alongside papers is not only good scientific standard but also essential for others to validate presented results and carry out comparisons. The last set of questions therefore concerns evaluation and reproducibility (ER), in particular, employed evaluation metrics as well as availability of data sets and source code.

ER-1: What log data sets are used for evaluating the approach?
ER-2: What evaluation metrics are employed?
ER-3: Does the evaluation consider runtime performance measurements?
ER-4: What approaches are used as benchmarks?
ER-5: Are the used data sets publicly available?
ER-6: Is the source code of the approach publicly available?

All aforementioned questions were assessed for each publication individually. The resulting feature matrix is presented in Table 2 in the following section and serves as the basis for our analyses and discussions.

4. Survey results

This section provides the assessments of all reviewed publications with respect to the features outlined in the previous section. We first provide some general information on the meta-data of publications before going over each reviewed feature in detail.

4.1. Bibliometrics

This section provides an overview of the distribution of publications per year as well as their citation counts.

4.1.1. Publications per year

Deep learning for anomaly detection in log data is a relatively new research field that has increasingly gained traction in the last years. Accordingly, a majority of the publications in this research area have only been published in the last two to three years. Fig. 2 shows an overview of the publication years of all publications reviewed for this survey. As expected, 58 out of the 62 considered publications were published in 2019 or later. As the search for relevant literature was carried out in the beginning of 2022, only two publications from that year are included. However, we expect to see an even higher number of publications in 2022 and beyond following the overall trend visible in the plot.

[8] Online or incremental processing refers to single-pass procedures where the runtime grows approximately linear with the number of processed lines.

[9] The output layer of the neural network comprises one or more nodes with certain numeric values.

Table 1
Top six most cited publications.

Citations	Approach	Year	Authors	Paper Title
963	DeepLog	2017	Du et al. (2017)	DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning
227	LogRobust	2019	Zhang et al. (2019)	Robust Log-Based Anomaly Detection on Unstable Log Data
209	LogAnomaly	2019	Meng et al. (2019)	LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs
92	–	2018	Lu, Wei, Li, and Wang (2018)	Detecting Anomaly in Big Data System Logs Using Convolutional Neural Network
53	Logsy	2020	Nedelkoski, Bogatinovski, Acker, Cardoso, and Kao (2020)	Self-Attentive Classification-Based Anomaly Detection in Unstructured Logs
45	LogBERT	2021	Guo, Yuan, and Wu (2021)	LogBERT: Log Anomaly Detection via BERT

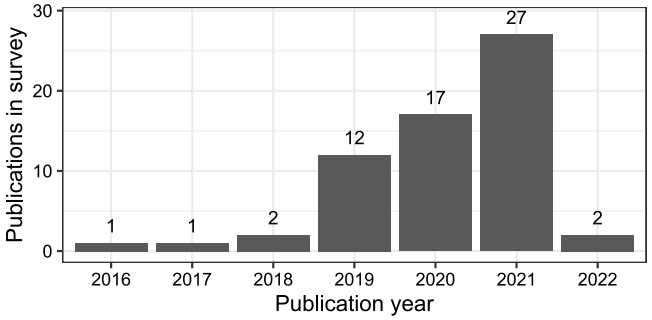


Fig. 2. Distribution of the number of publications per year.

4.1.2. Citations

Citation counts are a common indicator to assess the relevance and influence of publications. We therefore state the top six publication with the highest citation counts (according to Google Scholar) in Table 1. As of January 2023, the paper presenting DeepLog by Du, Li, Zheng, and Srikumar (2017) that was published in 2017 has the highest citation count and is arguably the most influential of all reviewed publications as they were the first to propose an approach based on deep learning that enables detection of anomalous event sequences in log data. Several of the subsequently published papers rely on the groundwork of DeepLog and it is therefore fair to assume that this paper is at least to some degree responsible for the increase of relevant publications from 2019 and onward that is visible in Fig. 2.

Note that the publication by Yang and Agrawal (2016) predates DeepLog (Du et al., 2017) but has a significantly lower citation count. The main reason for this is that the paper focuses on the analysis of tokens in single log events, a topic that received far less attention in subsequent research than the analysis of event sequences. This differentiation as well as assessments for all other features stated in Section 3.2 are presented in Table 2.

4.2. Deep learning techniques

This section provides an overview of the properties of deep learning models applied in reviewed publications.

4.2.1. Deep learning models

There are many different types of deep learning models (DL-1) that are suitable to be used for anomaly detection in log data (Sarker, 2021a, 2021b). The most basic form of a deep learning neural network is that of a **Multi-Layer Perceptron** (MLP), where all layers in the network are fully connected. Due to their simplicity, their classification accuracies are usually outperformed by other deep learning models that are specifically designed to capture common characteristics present in sequential data. Accordingly, they are rarely considered in the reviewed literature and only occur in combination with other deep learning models or as supplementary attention mechanisms (Li & Li, 2020; Wang, Zhang, Wang & Cao, 2021).

Convolutional Neural Networks (CNN) extend upon the architecture of MLPs by inserting convolutional and max pooling layers within the hidden layers. These layers enable that the neural networks capture more abstract features of the input data and at the same time reduce the input dimensions. This has proven especially effective when classifying 2-dimensional input data from images, where features such as lines are learned independent from their exact location in the image. This functionality is transferred to log data by arranging the log keys within a matrix so that the relationships between events, i.e., their temporal dependencies, are captured by the network (Lu et al., 2018). There also exist several approaches that rely on specific types of CNNs, such as temporal convolutional networks (TCN) that are specifically designed to process time-series and capture their short- and long-term dependencies through dilated causal convolutions (Wang et al., 2022; Zhang, Li, et al., 2021).

As visible in Table 2, **Recurrent Neural Networks** (RNN) are the most commonly used neural network architectures in the surveyed literature, with 36 out of 62 reviewed approaches leveraging RNNs for anomaly detection. The main reason for this is that the architecture of RNNs leverages feedback mechanisms that retain their states over time and thus directly enable learning of sequential event execution patterns in input data, which are the key identifiers for anomalies in log data sets that are commonly used in evaluations (cf. Section 4.5). Several different types of RNNs have successfully been applied for this purpose. One of the most widespread architectures are Long Short-Term Memory (LSTM) RNNs that are developed to enable long-time storage of states and comprise cells with input gates, output gates, and forget gates. A majority of the approaches leveraging LSTM RNNs train the network with sequences of event occurrences (or modified and enriched versions thereof) and subsequently disclose unusual sequential patterns in test data as anomalies (Du et al., 2017). Some approaches make use of Bi-LSTM RNNs, which are basically two independent LSTM RNNs that work in parallel and process sequences in opposite directions, i.e., while one LSTM RNN processes the input sequences as usual from the first to the last element, the other LSTM RNN processes sequence elements starting from the last entry and predicts elements that chronologically precede them. Experiments suggest that Bi-LSTM RNNs outperform LSTM RNNs (Farzad & Gulliver, 2019; Li et al., 2020; Ott et al., 2021; Sun et al., 2020; Syngal et al., 2021; Yu et al., 2021; Zhang, Dai, et al., 2021; Zhang et al., 2019). Another popular choice for RNNs are Gated Recurrent Units (GRU) that simplify the cell architecture as they only rely on update and reset gates. One of the main benefits of GRUs is that they are computationally more efficient than LSTM RNNs, which is a relevant aspect for use cases focusing on edge devices (Farzad, 2020; Farzad & Gulliver, 2019; Gu et al., 2021; Studiawan & Sohel, 2021; Sundqvist et al., 2020; Wang, Chen, et al., 2021; Xie et al., 2020; Yang & Agrawal, 2016; Yang et al., 2021).

While aforementioned deep learning models are primarily used for classification problems across different research fields, there are also models that are specifically designed to operate in unsupervised manner and are thus a natural choice for anomaly detection. One of them are **Autoencoders** (AE), which first create a code from the input data using an encoder and then try to approximate the input from the

Table 2
Survey results. DL-1: Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), Autoencoder (AE), Generative Adversarial Network (GAN), Transformer (TF), Attention mechanism (AT), Graph Neural Network (GNN), Evolving Granular Neural Network (EGNN); DL-2: Cross-Entropy (CE), Hyper-Sphere (HS), Mean Squared Error (MSE), Kullback–Leibler Divergence (KL), Marginal Likelihood (ML), Custom Loss Function (CF), Adversarial Training (AT), Not Available (NA); DL-3: Online (ON), Offline (OFF); DL-4: Supervised (SUP), Semi-supervised (SEMI), Unsupervised (UN); PP-1: Log key (KEY), Token (TOK), Combination (COM); PP-2: Token Sequence (TS), Token Count (TC), Event Sequence (ES), Event Count (EC), Parameter (PA), Event Interval Time (EI); PP-3: Event ID sequence (ID), Count Vector (CV), Statistical Feature Vector (FV), Semantic Vector (SV), Positional Embedding (PE), One-Hot Encoding (OH), Embedding Layer/Matrix (EL), Deep Encoded Embedding (DE), Parameter Vector (PV), Time Embedding (TE), Graph (G), Transfer Matrix (TM); AD-1: Outlier (OUT), Sequential (SEQ), Frequency (FREQ), Statistical (STAT); AD-2: Binary Classification (BIN), Input Vector Transformations (TRA), Reconstruction Error (RE), Multi-class Classification (MC), Probability Distribution (PRD), Numeric Vector (VEC); AD-3: Label (LAB), Threshold (THR), Highest Probabilities (TOP).

Approach	DL-1	DL-2	DL-3	DL-4	PP-1	PP-2	PP-3	AD-1	AD-2	AD-3	ER-6
Baril, Coustié, Mothe, and Teste (2020) (NoTIL)	RNN	CE	OFF	SEMI	KEY	ES, EC	CV	FREQ	VEC	THR	NO
Bursic, Cuculo, and D'Amelio (2019)	RNN, AE	MSE	OFF	UN	TOK	PA, TS	DE	OUT	RE	THR	NO
Catillo, Pecchia, and Villano (2022) (AutoLog)	AE	MSE	ON	SEMI	COM	STAT, TC	FV	FREQ	RE	THR	YES
Cheansunan and Phunchongharn (2019)	CNN	NA	OFF	SEMI	KEY	ES	EL	SEQ	PRD	TOP	NO
Chen, and Xiao and Jin (2021)	RNN, CNN	NA	OFF	SEMI	KEY	ES, EC	SV, CV	SEQ, FREQ	VEC, PRD	TOP	NO
Chen, Zhang, et al. (2020) (LogTransfer)	RNN, CNN	NA	OFF	SUP	KEY	ES	SV	SEQ	BIN	LAB	YES
Decker, Leite, Viola, and Bonacorsi (2020)	EGNN	CF	OFF	SUP	KEY	EC, STAT	FV	FREQ	MC	THR, LAB	NO
Du et al. (2017) (DeepLog)	RNN	CE, MSE	ON	SEMI	KEY	ES, PA, EI	PV, OH	SEQ	VEC, PRD	THR, TOP	RE
Du, Zhao, Xu, Han, and Zhang (2021) (LogAttention)	TF, AM	NA	OFF	SUP	KEY	ES	SV	SEQ	BIN	LAB	NO
Farzad and Gulliver (2019)	RNN, AE	CE	OFF	SUP	TOK	TS	EL	SEQ	BIN	LAB	NO
Farzad (2020)	RNN, AE, CNN, GAN	CE	OFF	SUP	TOK	TS	EL	OUT	BIN	LAB	NO
Farzad and Gulliver (2021)	RNN	CE	OFF	SUP	TOK	TS	EL	OUT	BIN	LAB	NO
Gu et al. (2021)	RNN, AM	CE	OFF	SEMI	KEY	ES	SV	SEQ	PRD	TOP	NO
Guo, Wu, Zhu, Yang, and Han (2021) (FLOGCNN)	CNN	CE	OFF	SUP	COM	TS	SV	SEQ	BIN	LAB	NO
Guo, Yuan, and Wu (2021) (LogBERT)	TF, AM	CE, HS	OFF	SEMI	KEY	ES	PE, EL	SEQ	PRD	TOP	YES
Guo, Lin, et al. (2021) (TransLog)	TF, AM	CE	OFF	SUP	KEY	ES	SV	SEQ	BIN	LAB	NO
Han and Yuan (2021) (LogTAD)	RNN, GAN	HS, CF, AT	OFF	UN	KEY	ES	SV	SEQ	TRA	THR	YES
Hashemi and Mäntylä (2021) (OneLog)	CNN	CE	OFF	SUP	TOK	ES	DE	SEQ	BIN	LAB	YES
Hirakawa, Tominaga, and Nakatoh (2020)	CNN, TF	CF	OFF	UN	TOK	ES	SV	OUT	BIN	THR	NO
Huang et al. (2020) (HitAnomaly)	TF, AM	CE	OFF	SUP	KEY	ES, PA	SV, PV	SEQ	BIN	LAB	NO
Le and Zhang (2021) (NeuralLog)	TF	CE	OFF	SUP	TOK	ES	SV, PE	SEQ	BIN	LAB	YES
Li and Li (2020) (LogSpy)	CNN, MLP, AM	CE	OFF	SUP	KEY	STAT	DE, EL	STAT	BIN	LAB	NO
Li, Chen, Jing, He, and Yu (2020) (SwissLog)	RNN, AM	CE	OFF	SUP	KEY	ES, EI	SV, TE	STAT, SEQ	BIN	LAB	NO
Liu et al. (2021) (LogNADS)	RNN	MSE	OFF	SUP	KEY	ES, PA	SV, PV	SEQ	BIN	LAB	NO
Lu et al. (2018)	CNN	NA	OFF	SUP	KEY	ES	EL	SEQ	BIN	LAB	NO
Lv, Luktarhan, and Chen (2021) (ConAnomaly)	RNN	NA	OFF	SUP	KEY	ES	SV	SEQ	BIN	THR	NO
Meng et al. (2019) (LogAnomaly)	RNN	NA	OFF	SEMI	KEY	ES, EC	SV, CV	SEQ, FREQ	VEC, PRD	TOP	RE
Nedelkoski et al. (2020) (Logsy)	TF	HS	OFF	UN	TOK	TS	PE, EL	OUT	TRA	THR	RE
Otomo, Kobayashi, Fukuda, and Esaki (2019)	AE	KL, ML, CF	OFF	SEMI	KEY	ES, EC	CV, OH	FREQ	TRA	THR	NO
Ott, Bogatinovski, Acker, Nedelkoski, and Kao (2021)	RNN	CE, MSE	OFF	SEMI	KEY	ES	SV	SEQ	VEC, PRD	THR, TOP	NO
Patil, Wadekar, Gupta, Vijan, and Kazi (2019)	RNN	CE	OFF	SUP	KEY	ES	OH	SEQ	BIN	LAB	NO
Qian, Ying, and Wang (2020) (VeLog)	AE	CF	ON	SEMI	KEY	ES, EC	ID, CV	SEQ, FREQ	RE	THR	NO
Studiawan and Sohel (2021)	AE	MSE	OFF	SEMI	KEY	EC, STAT, EI, TC	FV	STAT, FREQ	RE	THR	NO
Studiawan, Sohel, and Payne (2020) (pylogsentiment)	RNN	CE	OFF	SUP	TOK	TS	SV	OUT	BIN	LAB	YES
Sun et al. (2020) (AllContext)	RNN, AM	CE	OFF	SUP	KEY	ES	SV	OUT, SEQ	BIN, MC	LAB	NO
Sundqvist, Bhuyan, Forsman, and Elmroth (2020) (BoostLog)	RNN	NA	ON	SEMI	KEY	ES	ID	SEQ	PRD	THR	NO
Syngal, Verma, Karthik, Katyal, and Ghosh (2021)	RNN, AE	CE	OFF	SUP	TOK	ES, EC	CV, OH	SEQ, FREQ	RE	THR	NO
Wadekar, Gupta, Vijan, and Kazi (2019)	AE	CE, KL, ML	OFF	UN	KEY	ES	OH	SEQ	BIN	THR	NO
Wan, Liu, Wang, and Wen (2021) (GLAD-PAW)	GNN	CE	OFF	UN	KEY	ES	G	SEQ	PRD	TOP	NO
Wang, Xu, and Guo (2018)	RNN	NA	OFF	SUP	TOK	TS	SV	OUT	BIN	LAB	NO
Wang, Zhang, Wang and Cao (2021) (CATLog)	AE, MLP, TF	CE, CF	OFF	SUP	KEY	ES, TC	SV, DE, CV	SEQ, FREQ	BIN	LAB	NO
Wang, Tian, Fang, Chen, and Qin (2022) (LightLog)	CNN	CE	OFF	SUP	KEY	ES	SV	SEQ	BIN	LAB	YES
Wang, Chen, et al. (2021) (OC4Seq)	RNN	HS	OFF	UN	KEY	ES	EL	SEQ	TRA	THR	YES
Wibisono and Kistijantoro (2019)	TF, AM	NA	OFF	SEMI	KEY	ES	OH	SEQ	PRD	TOP	NO
Wittkopp et al. (2021) (A2Log)	TF, AM	CF	OFF	UN	TOK	TS	SV	OUT	BIN, TRA	THR	NO
Xi, Xin, Luo, Shang, and Tang (2021)	RNN, AM	CE	OFF	SEMI	KEY	ES	SV	SEQ	PRD	TOP	NO
Xia, Bai, Yin, Li, and Xu (2021) (LogGAN)	RNN, GAN	AT	ON	SEMI	KEY	ES	OH	SEQ	PRD	THR	NO
Xiao, Huang, and Wu (2019)	RNN, CNN	CE	OFF	SEMI	KEY	ES	EL	SEQ	PRD	TOP	NO
Xie, Ji, and Cheng (2020) (ATT-GRU)	RNN, AM	MSE	OFF	SEMI	KEY	ES, PA, EI	PV, EL	SEQ	VEC, PRD	THR, TOP	NO
Yang and Agrawal (2016)	RNN	CE	OFF	SEMI	TOK	TS	SV	OUT	VEC	THR	NO
Yang, Qu, Gao, Qian, and Tang (2019) (nLSALog)	RNN, AM	CE	OFF	SEMI	KEY	ES	EL	SEQ	PRD	TOP	NO
Yang et al. (2021) (PLELog)	RNN	NA	OFF	SEMI	KEY	ES	SV	SEQ	BIN	LAB	NO
Yen, Moh, and Moh (2019) (CausalConvLSTM)	RNN, CNN	CE	ON	SEMI	KEY	ES	OH	SEQ	PRD	TOP	NO
Yin et al. (2020) (LogC)	RNN	CE	OFF	SEMI	KEY	ES, PA	OH	SEQ	PRD	TOP	NO
Yu et al. (2021)	RNN	CE	OFF	SEMI	KEY	ES, EC	SV, ID, CV	SEQ, FREQ	PRD	TOP	NO
Zhang et al. (2019) (LogRobust)	RNN, AM	CE	OFF	SUP	KEY	ES	SV	SEQ	BIN	LAB	RE
Zhang, Li, et al. (2021) (LogAttn)	AE, CNN, AM	CE	OFF	SEMI	TOK	ES, EC	SV, CV	SEQ, FREQ	RE	THR	NO
Zhang, Wang, Zhang, Zhang, and Han (2021) (LSADNET)	CNN, TF	CE	ON	SEMI	KEY	ES, STAT	SV, TM	STAT, SEQ	PRD	TOP	NO
Zhang, Dai, Han, and Zheng (2021) (SentiLog)	RNN	NA	OFF	SUP	TOK	TS	SV	OUT	BIN	LAB	NO
Zhao, Niu, et al. (2021) (Trine)	TF, GAN	NA	OFF	SEMI	TOK	ES	SV, DE, PE	SEQ	BIN	LAB	NO
Zhou et al. (2020) (LogSayer)	RNN, CNN	NA	ON	SUP	KEY	EC, STAT	FV	STAT	BIN	LAB	NO
Zhu, Li, Gu, and Wang (2020) (LogNL)	RNN	NA	OFF	SEMI	KEY	ES, PA, EI	SV, PV	SEQ	VEC, PRD	THR	NO

code using a decoder, thereby avoiding the need for labeled input data. The main idea is that through this process the neural network learns the main features from the input but neglects the noise in the data, similar to dimension reduction techniques such as principal component analysis (PCA). Any input data that is fed into an already trained network and yields a high reconstruction error is then considered as anomalous. Besides the standard model for Autoencoders, there are also several related types, such as Variational Autoencoders (VAE) that operate on statistical distributions (Qian et al., 2020; Wadekar et al., 2019; Wang, Zhang, Wang & Cao, 2021), Conditional Variational Autoencoders (CVAE) that add conditional information such as event types to the training (Otomo et al., 2019), and Convolutional Autoencoder (CAE) that leverage the advantages of CNNs regarding learning of location-independent features (Wadekar et al., 2019).

Generative Adversarial Networks (GAN) are another approach for unsupervised deep learning. They actually consist of two separate components that compete with each other: a generator that produces new data that resembles the input data, and a discriminator that estimates the probability that some data stems from the input data, which is used to improve the generator. Existing approaches use different models to construct GANs, including LSTM RNNs (Han & Yuan, 2021; Xia et al., 2021), CNNs in combination with GRUs (Farzad, 2020), and Transformers (Wang, Zhang, Wang & Cao, 2021; Zhao, Niu, et al., 2021).

Transformers (TF) make use of so-called self-attention mechanisms to embed data instances into a vector space, where similar instances should be closer to each other than dissimilar ones (Du et al., 2021; Guo, Yuan, & Wu, 2021; Nedelkoski et al., 2020). The goal of Transformers is to assign weights to specific inputs according to the context of their occurrence, such as words in sentences. Accordingly, Transformers have been particularly successful in the area of natural language processing (NLP). **Attention mechanisms** (AT) do not just appear in Transformers, but are also frequently used to improve classification and detection in other deep neural networks such as RNNs by weighting relevant inputs higher. This effect is particularly strong when long sequences are ingested by RNNs (Zhang, Li, et al., 2021). Attention mechanisms are usually realized as trainable networks such as MLPs (Li & Li, 2020). In order to avoid confusions with the Transformer model, we state these additional attention mechanisms explicitly in Table 2.

Table 3
Definitions of common training loss functions.

Name	Description	Equation
Cross-Entropy	Loss between a ground truth label y and the predicted label $\hat{y}$.	$H(y,\hat{y}) = -\sum_{j=1}^{N} y_j log\left(\hat{y}_j\right)$
Hyper-Sphere Objective Function	Distance between embedding vector y and hyper-sphere center c.	$L_{HS} = \frac{1}{N}\sum_{j=1}^{N}\left\|y_j - c\right\|^2$
Mean Squared Error	Loss between a ground truth label y and the predicted label $\hat{y}$.	$L_{MSE}(y,\hat{y}) = \frac{1}{N}\sum_{j=1}^{N}\left(y_j - \hat{y}_j\right)^2$
Kullback–Leibler Divergence	Statistical divergence between probability distributions P and Q.	$KL(P\|Q) = \sum_{x\in\mathcal{X}} P(x) log\left(\frac{P(x)}{Q(x)}\right)$
Adversarial Training Function	Loss for vector y and prediction $\hat{y}$ with generator G and discriminator D.	$L_{AT} = \min_G \max_D \left(\mathbb{E}\left(log\left(D(G(\hat{y}))\right)\right) + \mathbb{E}\left(log\left(1 - D(G(y))\right)\right)\right)$

Wan et al. (2021) are the only authors to utilize **Graph Neural Networks** (GNN). While neural networks typically ingest ordered data, e.g., CNNs rely on 2-dimensional input data and RNNs require sequences of observations, GNNs are designed to ingest graph inputs, i.e., sets of vertices and edges. One possibility to transform log data into graphs is to generate session graphs, with vertices representing events and edges their sequential executions. Another less commonly applied type of deep learning model is the **Evolving Granular Neural Network** (EGNN), a fuzzy inference system that is gradually constructed from online data streams (Decker et al., 2020).

Our survey shows that many of the reviewed approaches rely on only one type of deep learning model. However, some authors also use combinations of different models. For example, Wang, Zhang, Wang and Cao (2021) propose to use a MLP to combine the output of a VAE with that of a Transformer trained with adversarial learning.

4.2.2. Training loss function

Loss functions (DL-2) are essential for training of deep neural networks as they quantify the difference between the output of the neural network and the expected result (Zhang & Sabuncu, 2018). The most common loss function in the reviewed publications is the **Cross-Entropy** (CE), in particular, the categorical cross-entropy for multi-class prediction (Du et al., 2017; Syngal et al., 2021) or binary cross-entropy that only differentiates between the normal and anomalous class (Wang et al., 2022). Other common loss functions include the **Hyper-Sphere Objective Function** (HS) where the distance to the center of a hyper-sphere represents the anomaly score (Guo, Yuan, & Wu, 2021; Han & Yuan, 2021; Nedelkoski et al., 2020; Wang, Chen, et al., 2021), the **Mean Squared Error** (MSE) that is used for regression (Bursic et al., 2019; Catillo et al., 2022; Du et al., 2017; Liu et al., 2021; Ott et al., 2021; Studiawan & Sohel, 2021; Xie et al., 2020), and the **Kullback–Leibler Divergence** (KL) and its extension **Marginal Likelihood** (ML) that are useful to measure loss in probability distributions (Otomo et al., 2019; Wadekar et al., 2019).

Some of the presented approaches are trained with **Custom Loss Functions** (CF), including combinations of CE and HS (Wittkopp et al., 2021). Some authors also define objective functions specifically for **Adversarial Training** (AT) of GANs (Han & Yuan, 2021; Xia et al., 2021). Table 3 summarizes the main loss functions. Out of all publications, 14 do not state the loss function and are therefore marked as **Not Available** (NA).

4.2.3. Operation mode

When applying anomaly detection in real world scenarios, not all log data is available at any time; instead, events are generated as a continuous stream and should be analyzed only at their time of occurrence in order to enable (close to) real-time detection. Thereby, the structural integrity and statistical properties of the generated logs vary over time, e.g. the overall system utilization is not stationary as user interactions and the technological environment are subject to change. In addition, log templates change or new log events occur when applications generating these logs are modified (Zhang et al., 2019).

To keep up with these changes in an automated way, algorithms need to adopt online or incremental learning (DL-3), i.e., ingest input data with linear time complexity so that data sets are processed in a single pass where each data instance is only handled once. While it is usually always possible to carry out detection in an online fashion when trained models are considered static, it is significantly more challenging to develop algorithms that support online learning and dynamically update their models to adapt to new events or patterns by incorporating them into the baseline for detection (Cui, Ahmad, & Hawkins, 2016; Hadsell, Rao, Rusu, & Pascanu, 2020). As continuous learning is still an open problem, we mark all approaches that enable dynamic model updates at least to some degree as **online** (ON) and all other approaches with offline training phases as **offline** (OFF) in Table 2.

The reviewed approaches addressed dynamic model adaptation in multiple ways. Du et al. (2017) suggest to update the weights of their neural network when false positives are identified during the detection to reflect the correct event probability distributions without the need to re-train from scratch. Meng et al. (2019) argue that such a manual feedback loop is infeasible in practice and therefore resort to re-training where new event types are mapped to existing ones. Yen et al. (2019) automatically re-train their neural network on batches of new log data when false positive rates exceed a certain threshold. However, calculating the false positive rates relies on labeled data and thus does not remove the human from the loop. A promising solution to aforementioned problems is presented by Decker et al. (2020), who employ an evolving classifier that is specifically designed to handle unstable data streams and could thus enable continuous learning.

In general, both online and offline models can work in **supervised** (SUP) as well as **unsupervised** (UN) fashion (DL-4). However, we noticed that almost all supervised learning approaches in the reviewed publications opt for an offline training phase. This is reasonable as the label information required for supervised learning relies on manual analysis or validation and therefore can only be generated forensically for delimited data sets, but not for data streams.

We also made the observation that many reviewed approaches claim to enable unsupervised learning, but are in fact operated in **semi-supervised** (SEMI) fashion as they usually assume that training takes place only on normal data that is free of anomalies (Chen, and Xiao & Jin, 2021). The main problem is that anomalies that are present in training data would incorrectly change the weights of the neural networks and thus deteriorate their detection in the subsequent detection phase. Accordingly, only deep learning models that are designed for unsupervised learning are capable of handling anomalies in training data, for example, the approach based on a CVAE model presented by Otomo et al. (2019). Note that neural network architectures that would support unsupervised learning may also be applied for semi-supervised detection and are therefore not necessarily marked as unsupervised.

4.3. Log data preparation

This section outlines all steps necessary to prepare raw and textual log data for deep learning. This includes grouping of events into windows or sessions, tokenization and log parsing, extraction of features from the logs, as well as transformation of these features into vector

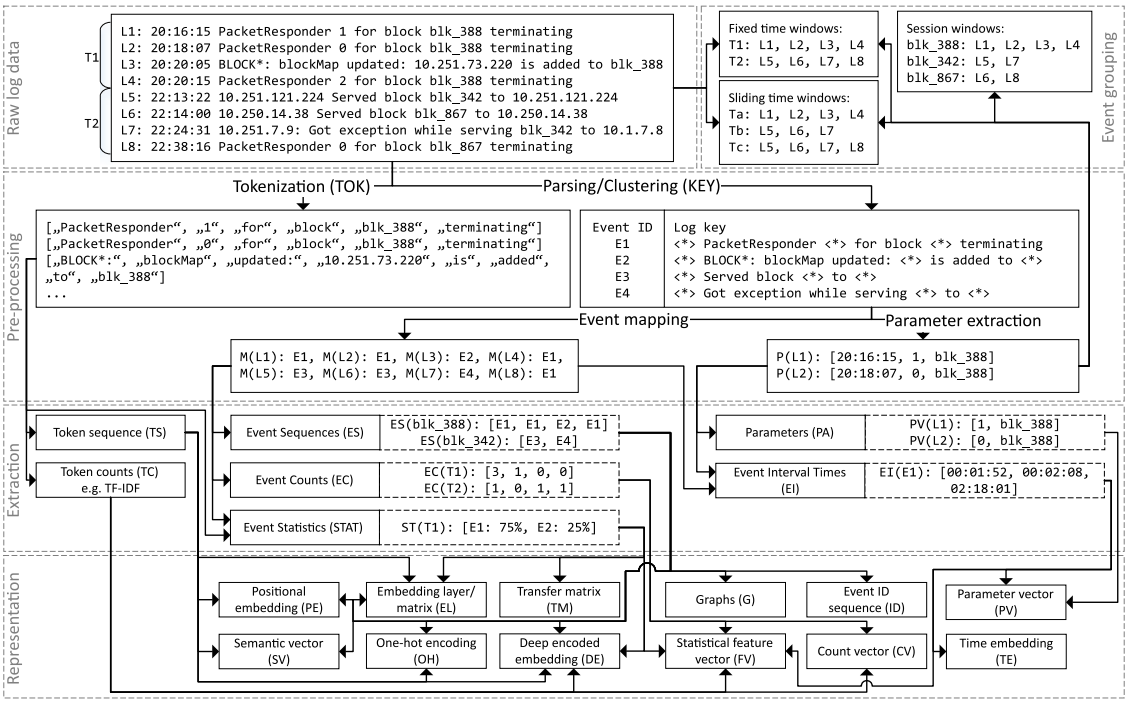


Fig. 3. Different stages of data preparation to transform raw log data into numeric vectors.

representations that are suitable as input to neural networks. Fig. 3 provides an overview of these data preparation methods used in the reviewed literature and illustrates them on the sample log lines L1–L8. In the following sections, we discuss all stated methods in detail.

4.3.1. Pre-processing

As log data is generally unstructured it is necessary to pre-process them in some way before feeding them into deep learning systems (PP-1). Our survey shows that there are two main approaches to handle unstructured data. The most common approach is to leverage parsers, which are usually referred to as **log keys** (KEY), to extract unique event identifiers for each line as well as event parameter values as described in Section 2.1.2. Thereby, authors usually re-use existing log keys for well-known data sets or create the keys using state-of-the-art approaches for parser generation, such as Drain (He, Zhu, Zheng, & Lyu, 2017) or Spell (Du & Li, 2016). Typically, each line matches exactly one key; it is thus easy to assign a unique event type identifier to each log line during event mapping. For example, in Fig. 3 line L1 is mapped to event type identifier E1 as it matches the corresponding log key. Moreover, the figure also shows that parsing allows to extract all parameters from log events, in particular, the time stamp and identifiers for packet responder and file block are extracted from L1 and stored in a list.

An alternative to parsing are **token-based** (TOK) strategies that split log messages into lists of words, for example, by splitting them at white spaces. It is then common to clean the data by transforming all letters to lowercase and removing special characters and stop words before obtaining the final word vectors. While such approaches draw less semantic information from the single tokens, they have the advantage of being more flexible as they rely on generally applicable heuristics rather than pre-defined parsers and are therefore widely applicable. Some approaches make use of a **combination** (COM) of parsing and token-based pre-processing strategies, in particular, by generating token vectors from parsed events rather than raw log lines (Catillo et al., 2022; Guo, Wu, et al., 2021; Le & Zhang, 2021).

4.3.2. Event grouping

As pointed out in Section 2.1.3, simple outlier detection does not require any grouping of logs since single unusual events are regarded as anomalies independent from the context in which they occur in. However, deep learning is most often applied to disclose unusual patterns of multiple log events, such as changes of event sequences or temporal log correlations. For these cases it is necessary to logically organize events into groups that are then analyzed individually or in relation to each other.

We illustrate common event grouping strategies in the top right of Fig. 3. Grouping into time windows is almost always feasible as log events are generally produced with a time stamp that documents their time of generation (Landauer, Skopik, Wurzenberger, & Rauber, 2020). Since time stamps commonly occur in the beginning of log lines and are thus relatively easy to extract, it is not necessary to parse the remainder of the usually more complex log messages. There are two main strategies for time-based grouping (He et al., 2016). First, **sliding time windows** are windows of a specific duration that are shifted across the log data with a fixed step size that is generally smaller and a whole number divisor of the window size. For every step where the time window is moved all logs with a time stamp within the current start and end time of the window are allocated to the same group, where each log line may appear in multiple groups as time windows overlap. For the sample logs in Fig. 3 we assume time window of 2 hours starting at 20:00:00 and a step size of 30 minutes. As visible in the figure, lines L5, L6, and L7 are contained in both Tb and Tc. Second, **fixed time windows** are special cases of sliding time windows where the step size is set to the same value as the window size. While this strategy results in a less fine-grained view on the data, the advantage is that each log event will only be allocated to exactly one time window, which makes subsequent computations such as time-series analysis easier. The log events in Fig. 3 are exemplarily grouped into fixed time windows of 2 hours, yielding $T1$ containing the first set of four lines and $T2$ containing the subsequent set of four lines. It must be noted that grouping based on sliding or fixed windows may also be carried out by numbers of lines rather than time so that each group of lines has the same size. While this avoids the need to process time stamps and ensures that the group sizes are fixed (e.g., there cannot be any empty groups), it is more difficult to consider event frequencies as time-series as the resulting windows represent varying time spans.

An entirely different grouping strategy are **session windows** that rely on an event parameter that acts as an identifier for a specific task

or process where the event originated from. Grouping log events by these session identifiers allows to extract event sequences that correctly depict underlying program workflows even when multiple sessions are carried out in parallel on the monitored system. Unfortunately, not all types of log data come with such an identifier for sessions. In the sample logs depicted in Fig. 3 the file block identifier (e.g., blk_388) acts as a session identifier and thus allows us to exemplarily extract three event sequences.

4.3.3. Feature extraction

The parsing- and token-based pre-processing strategies described in Section 4.3.1 enable the extraction of structured features from the otherwise unstructured logs (PP-2). Some of these features are directly derived from pre-processing logs, e.g., the **Token Sequence** (TS) may be used without any further modifications for analyzing each log line as a sentence of words. On the other hand, **Token Counts** (TC) require an additional step of computation where the tokens in each line are compared and counted, including advanced weighting techniques for each token such as term frequency-inverse document frequency (TF-IDF) that estimates token relevance based on token occurrences across all observed log lines.

Considering the outcome of the event mapping step it is simple to extract **Event Sequences** (ES), i.e., sequences of event type identifiers that are usually separated by fixed, sliding, or session windows (cf. Section 4.3.2). For example, Fig. 3 shows that the event sequence for file block blk_388 is $[E1, E1, E2, E1]$ corresponding to the event identifiers for the respective log keys matching the lines L1–L4. **Event Counts** (EC) are vectors of length d, where the ith element of the vector depicts the number of occurrences of the ith log key and d is the total number of available log keys. The example in Fig. 3 shows the event count vector for time window $T1$ as $[3, 1, 0, 0]$, indicating that three log lines corresponding to the first log key (E1) appeared in $T1$, in particular, the lines L1, L2, and L4. Besides frequencies, many other **statistical properties** (STAT) may be computed from event occurrences, such as the percentage of seasonal logs (Zhou et al., 2020), the lengths of log messages (Studiawan & Sohel, 2021), log activity rates (Decker et al., 2020), entropy-based scores for chunks of log lines (Catillo et al., 2022), or the presence of sudden bursts in event occurrences (Zhou et al., 2020).

Parameters (PA) of log events are extracted as lists of values. Since the semantic meaning of each parameter is known after parsing, the values in each vector can be analyzed with methods that are appropriate for the respective value types, e.g., numeric or categorical. One special parameter of log events is the time stamp as it allows to put event occurrences into chronological order and infer dynamic dependencies. Accordingly, **Event Interval Times** (EI), i.e., inter-arrival times of log lines that belong to the same event type, are a frequently extracted feature for anomaly detection.

4.3.4. Feature representation

The extracted features described in the previous section comprise numeric or categorical vectors and are suitable to be consumed by neural networks. For example, event sequences are represented as **Event ID sequence vectors** (ID), i.e., chronologically ordered sequences of log key identifiers, and fed into RNNs to learn dependencies of event occurrences and disclose unusual sequence patterns as anomalies (Sundqvist et al., 2020). Event counts are represented as ordered **Count Vectors** (CV) and are also similarly used as input for RNNs (Baril et al., 2020). Event statistics are another type of input that do not require any specific processing other than representing them in a **Statistical Feature Vector** (FV) where the position of each element in the vector corresponds to one particular feature.

While it is possible to directly use the extracted features, most of the approaches presented in the reviewed publications in fact rely on combinations or otherwise transformed vector representations of the original feature vectors (PP-3). Thereby, the most common representation is that of a **Semantic Vector** (SV). Within the field of NLP it is common practice to transform words of a sentence into so called semantic vectors that encode context-based semantics (e.g. Word2Vec (Mikolov, Chen, Corrado, & Dean, 2013), BERT (Devlin, Chang, Lee, & Toutanova, 2018), or GloVe (Pennington, Socher, & Manning, 2014)) or language statistics (e.g. TF-IDF (Manning, Raghavan, & Schütze, 2010)). Since each log line comprises sequences of tokens analogous to words in a sentence, it stands to reason to apply methods from natural language processing on the token sequences. Similarly, a sequence of multiple subsequent events can be regarded as a sentence, where each unique log key represents a word. Semantic encoding is typically achieved by training deep neural methods on a specific log file or by relying on pre-trained models. Semantic vectors are sometimes used in combination with **Positional Embedding** (PE), where elements (typically tokens) are encoded based on their relative positions in a sequence. To add the positional information to the encoded log messages authors usually use sine and cosine functions for even and odd token indices respectively (Le & Zhang, 2021; Nedelkoski et al., 2020; Zhao, Niu, et al., 2021).

One-Hot Encoding (OH) is one of the most common techniques to handle categorical data and is therefore frequently applied on event types (as a finite number of log keys is defined in the parser) or token values. Formally, the one-hot encoding of a value i from an ordered list of d values is a vector of length d where the ith element is 1 and all others are 0 (Du et al., 2017). While most authors use one-hot encoded data directly as an input to neural networks, it is also possible to combine it with other features such as count vectors so that the applied neural network is capable of discerning the input and learn separate models for different log keys (Otomo et al., 2019).

Embedding Layers/Matrices (EL) are typically used to resolve the problems with respect to sparsity of high-dimensional input data such as one-hot encoded event types when a large number of log keys are required to parse the logs (Wang, Chen, et al., 2021; Yang et al., 2019). They are usually randomly initialized parameters which are trained alongside the classification models to create optimal vector encodings for log messages (Lu et al., 2018). The vector encodings are usually arranged in a matrix so that the respective vector for a particular log key is obtained by multiplying the matrix with a one-hot encoded log key vector. The main difference to semantic vectors is that embedding layers/matrices are generally not trained towards NLP objectives, i.e., they do not aim to learn the semantics of words like Word2Vec; instead, embedding layers/matrices are only trained to minimize the loss function of the classification network. Some authors also use custom embedding models based on deep learning; we refer to their output as **Deep Encoded Embeddings** (DE). This includes a combination of character-, event- and sequence-based embeddings (Hashemi & Mäntylä, 2021), attention mechanisms using MLPs and CNNs (Li & Li, 2020), and token counts with label information fed into VAEs (Wang, Zhang, Wang & Cao, 2021).

Other than aforementioned methods that operate with event types and mostly use tokens only for encoding these events in vector format, approaches that rely on **Parameter Vectors** (PV) directly use the actual values extracted from the parsed log messages. Thereby, extracted parameters that are numeric may be used for multi-variate time-series analysis with RNNs (Du et al., 2017; Xie et al., 2020; Zhu et al., 2020), while categorical values could be one-hot encoded and vectorized with word embedding methods (Huang et al., 2020). Either way, as different log events have varying numbers of parameters with different semantic meaning, it is usually necessary to analyze the parameters of each event type on their own (Du et al., 2017; Xie et al., 2020; Zhu et al., 2020). The time stamp of log events is a special parameter as it allows to put other parameters in temporal context, which is required for time-series analysis. However, the time stamps themselves may be used for **Time Embedding** (TE) and serve as input to neural networks (Bursic et al., 2019). For this purpose, Li et al. (2020) generate vectors for sequences

of time differences between event occurrences by applying soft one-hot encoding.

While aforementioned representations are used in different variations in several publications, **Graphs** (G) are an entirely different approach that is only employed by Wan et al. (2021). The key idea is to transform event sequences into session graphs and then apply neural networks that are specifically designed for these data structures. Another less common strategy for encoding dependencies between log messages is the so-called **Transfer Matrix** (TM). In particular, the $d \times d$-dimensional matrix encodes the probabilities that any of the d log keys follows another (Zhang, Wang, et al., 2021).

4.4. Anomaly detection techniques

The previous sections described methods to prepare the input log data for ingestion by neural networks. However, in many cases it is non-trivial to retrieve the information whether a data sample presented to the neural network is anomalous or not as there are no dedicated output nodes for anomalies that are not known beforehand. This section therefore outlines the anomaly detection techniques applied by the reviewed approaches. First, we state different types of anomalies that are commonly targeted by approaches. Second, we investigate how the detection or classification result is retrieved from the network output. Finally, we state different decision criteria that are used to differentiate normal from anomalous samples based on these resulting measures.

4.4.1. Anomaly types

The reviewed approaches address different types of anomalies as outlined in Section 2.1.3 (AD-1). **Outliers** (OUT) are single log events that do not fit to the overall structure of the data set. Most commonly, outlier events are detected based on their unusual parameter values (Hirakawa et al., 2020), token sequences (Wang et al., 2018; Wittkopp et al., 2021), or occurrence times (Bursic et al., 2019). Comparatively few approaches consider outliers since the majority of reviewed approaches focus on collective anomalies, in particular, involving sequences of events.

Sequential (SEQ) anomalies are detected when execution paths change, i.e., the applications that generate logs execute events differently than before. This could result in additional, missing, or differently ordered events within otherwise normal event sequences as well as completely new sequences that could even involve previously unseen event types. A common method to detect these anomalies is to check whether a specific event type in a sequence of events is expected to occur given all the events that occur before or thereafter.

While the detection of sequential anomalies inherently assumes that events occur as ordered sequences, **frequency** (FREQ) anomalies only consider the number of event occurrences. Nonetheless, event frequencies may be used to infer dependencies between events, for example, the numbers of events related to opening and closing files should be the same as every file will eventually be closed and needs to be opened before doing so (Meng et al., 2019). The main idea for detecting frequency anomalies is that changes of system behavior affect the number of usual event occurrences that are most often counted within time windows.

Some approaches also consider anomalies that base on certain quantitatively expressed properties of multiple log events that go beyond event counts, such as their inter-arrival times (Li et al., 2020) or seasonal occurrence patterns (Zhou et al., 2020). We refer to them as **statistical** (STAT) anomalies, because their detection generally assumes that the event occurrences follow specific stable distributions over time. The following section describes how the output of the neural networks is used to determine the aforementioned types of anomalies.

4.4.2. Network output

In general, the output of a neural network consists either of a single node or multiple nodes in its final layer (AD-2). Accordingly, the resulting value extracted from the network is a scalar or vector of numeric values. One possibility is to consider these results as an anomaly score that expresses to what degree the log events presented to the network represent an anomaly or not. As these scores are generally difficult to interpret on their own, it is usually necessary to compare them with some threshold. In **binary classification** (BIN) this idea is used to estimate whether the input presented to the neural network is either normal or anomalous. For supervised approaches the numeric output can be interpreted as probabilities that the input corresponds to either class. On the other hand, anomaly scores that are generated by semi- or unsupervised approaches are generally not normalized, e.g., the distance between the input and the center of a hyper-spherical cluster can become arbitrarily large, and therefore need to be compared to empirically determined thresholds. Similarly, **Input vector transformations** (TRA) that transform the input into a new vector space and generate clusters for normal data are capable of detecting outliers by their large distances to cluster centers. Another related method is to leverage the **reconstruction error** (RE) of Autoencoders that first encode the input data in a lower dimensional space and then attempt to reconstruct them to their original form. In this case, input samples are considered anomalous if they are difficult to reconstruct, i.e., yield a large reconstruction error, because they do not correspond to the normal data that the network was trained on.

While aforementioned approaches pursue binary classification that separates normal from anomalous input, there are also concepts that are capable of differentiating between more than two classes. **Multi-class classification** (MC) assigns distinct labels to specific types of anomalies but requires supervised learning in order to capture the patterns specific to these classes in the training phase. To resolve this issue, it is also possible to consider event types as the target of classification. The most common approach for this is to train the models to predict the next log key following a sequence of observed log events. When a softmax function is used as an activation for the output, this prediction yields a **probability distribution** (PRD) with the individual probabilities for each log key. The problem of predicting the next log event in a sequence can also be formulated as a regression task when events are considered as **numeric vectors** (VEC), in particular, semantic or parameter vectors. Thereby, the neural network outputs a vector representing the expected event vector instead of the respective event type.

4.4.3. Detection method

The different strategies for obtaining the network output described in the previous section already give a rough idea on the methods used to differentiate normal from anomalous behavior and eventually report anomalies (AD-3). When the network output directly corresponds to a particular **label** (LAB), for example, as accomplished by binary classification, it is simple to generate anomalies for all samples that are labeled as anomalous. For all approaches that output some kind of numeric value or anomaly score it is straightforward to use a **threshold** (THR) for differentiation. This threshold is also useful to tune the detection performance of the approach and find an acceptable tradeoff between TPR and FPR by empirical experimentation. Another approach is to model the anomaly scores obtained from the network as statistical distributions. In particular, Du et al. (2017) and Xie et al. (2020) use a Gaussian distribution to detect parameter vectors with errors outside of specific confidence intervals as anomalous. A different approach is proposed by Otomo et al. (2019), who apply clustering on the reconstruction errors and detect all samples that are sufficiently far away from the normal clusters as anomalies.

When the output of the neural network is a multi-class probability distribution for known log keys it is common to consider the top n log keys with the **highest probabilities** (TOP) as candidates for

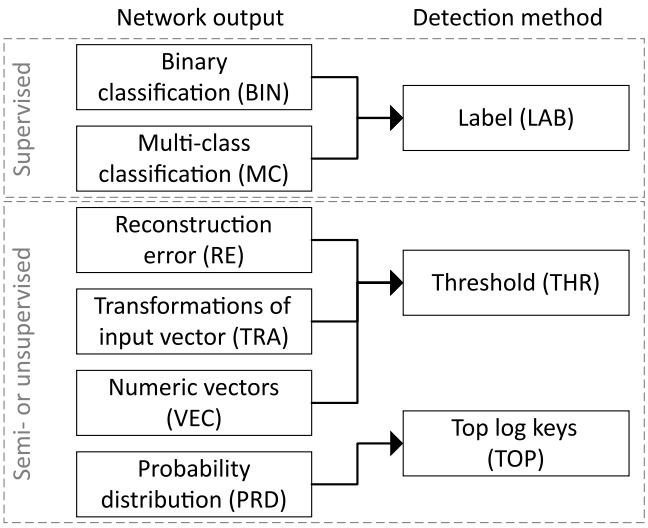


Fig. 4. Most common combinations of network output types and detection techniques.

classification. Thereby, an anomaly is only detected if the actual type of the log event is not within the set of candidates. The number of considered candidates n regulates the tradeoff between TPR and FPR analogous to the aforementioned threshold. Fig. 4 shows how the network output relates to the applied detection techniques. Both BIN and MC rely on supervised learning and are therefore able to directly assign labels to new input samples. All other techniques enable semi- or unsupervised training. In particular, RE, TRA, and VEC produce anomaly scores that are compared against thresholds, while PRD is typically compared against the top log keys with highest probabilities. Note that there are some exceptions to this overall pattern. For example, the approach by Yang et al. (2021) supports semi-supervised training through the use of probabilistic labels, and the approaches by Zhang et al. (2019) and Syngal et al. (2021) are supervised despite relying on reconstruction errors.

4.5. Evaluation & reproducibility

This section provides an overview of evaluations carried out in the reviewed publications. We present a list of openly available data sets that are useful for evaluations and state commonly used evaluation metrics and benchmarks. We also discuss reproducibility of evaluations with respect to the availability of open-source implementations.

4.5.1. Data sets

Data sets are essential in scientific publications to validate the approach and show improvements over state-of-the-art detection rates (ER-1). Our literature review reveals that there are only few data sets that are commonly used when evaluating log data anomaly detection approaches using deep learning. Table 4 shows an overview of all data sets used in the reviewed publications. As visible in the table, the vast majority of evaluations rely on only four data sets (HDFS, BGL, Thunderbird, and OpenStack). In the following, we briefly describe each data set.

The **HDFS** log data set stems from the Hadoop Distributed File System (HDFS) running on a high-performance computing cluster with 203 nodes that computes many standard MapReduce jobs. More than 24 million logs are collected over a period of two days. The data set comprises sequences of heterogeneous log events for specific file blocks that act as identifiers for sessions. Some of the event sequences correspond to anomalous execution paths that are mostly related to performance issues such as write exceptions, which were manually labeled (Xu, Huang, Fox, Patterson, & Jordan, 2009a). The sample logs shown in Fig. 3 are simplified versions of the logs from the HDFS data set.

The **BGL** data set comprises more than four million log events that were collected over more than 200 days from a BlueGene/L (BGL) supercomputer running at the Lawrence Livermore National Labs. The log events were generated with a severity field that allows to separate them into classes; however, the logs were additionally labeled manually by system administrators. The anomalies occurring in the logs correspond to both hardware and software problems. Other log data sets from the same family that are also presented in the study by Oliner and Stearley (2007) are **Thunderbird** and **Spirit**. These data sets comprise system logs (syslog) and similarly comprise alerts related to system problems such as disk failures. The events in both data sets include automatically generated alert tags that can be used as labels. The **HPC RAS** data set (Zheng et al., 2011) is another log data set from the same family of supercomputers. Reliability, Availability, and Serviceability (RAS) logs are usually used to understand the reasons for system failure. However, this log data set does not include labels for anomalies.

The **OpenStack** data set was collected from an OpenStack platform where automatic scripts continuously and randomly carry out tasks related to handling of virtual machines, including creation, pausing, deletion, etc. Other than aforementioned data sets that mostly comprise randomly occurring failures, the authors of the OpenStack data set purposefully injected anomalies at specific points in time, including timeouts and errors. The events include instance identifiers that can be used to identify sessions (Du et al., 2017).

Similar to the HDFS data, the **Hadoop** data set comprises logs from a computing cluster that runs the MapReduce jobs WordCount and PageRank. After an initial training phase, the authors purposefully trigger failures on the nodes, in particular, by shutting down a machine, disconnecting the network, and filling up the hard disk of one server. The logs are separated into different files according to application identifiers that act similar to session identifiers and are also used by the authors to assign labels to anomalous program executions (Lin et al., 2016).

Loghub (He et al., 2020) comprises several data sets that are used in evaluations, including logs from high-performance computing systems. The **Spark** data set contains logs from the distributed data processing engine Apache Spark running on 32 machines. The logs include normal and anomalous executions, but are not labeled. **ZooKeeper** is another Apache service used in distributed computing and configuration management. Loghub also comprises logs from conventional operating systems. The **Windows** log data set was collected on a laboratory Windows 7 machine by monitoring CBS (Component Based Servicing), which operates on a package and update level. Similarly, the **Android** data set was collected from a mobile phone in a laboratory setting. Both data sets are not labeled and do not involve any purposefully injected anomalies. Logs from the Linux operating system are provided in the disk images of the **Digital Corpora**, where failures such as invalid authentications occur in the data (Garfinkel et al., 2009).

When it comes to the detection of anomalous behavior, all aforementioned data sets mainly provide failure events that are generated as part of legitimate system usage. However, there are also data sets that instead involve events generated from malicious activities and thus enable evaluation of anomaly-based intrusion detection systems. For example, the **DFRWS 2009** data set contains system logs from Linux devices that involve data exfiltration, unauthorized accesses, as well as the use of backdoor software (Eoghan & Golden, 2022). The **Honeynet 2010** (Marty et al., 2022) and **Honeynet 2011** (Arcas et al., 2022) data sets comprise common log files from compromised Linux machines that were illegitimately accessed. The Public Security Log Sharing Site (Chuvakin, 2010) provides **Linux Security Logs** from diverse sources and affected by different real-world intrusions, such as brute-force attacks. Unfortunately, none of these security log files come with labels for malicious events and thus authors need to generate their own ground truths to evaluate their approaches on these data sets (Studiawan & Sohel, 2021).

Table 4
Common public log data sets.

Data set	Use-case	Labels	Sessions	Anomaly sources	Used in evaluation
HDFS (Xu et al., 2009b)	High-perf. comp.	✓	✓	Failures	Bursic et al. (2019), Cheansunan and Phunchongharn (2019), Chen, and Xiao and Jin (2021), Chen, Zhang, et al. (2020), Du et al. (2017, 2021), Gu et al. (2021), Guo, Lin, et al. (2021), Guo, Wu, et al. (2021), Guo, Yuan, and Wu (2021), Hashemi and Mäntylä (2021), Huang et al. (2020), Le and Zhang (2021), Li et al. (2020), Liu et al. (2021), Lu et al. (2018), Lv et al. (2021), Meng et al. (2019), Patil et al. (2019), Qian et al. (2020), Sun et al. (2020), Sundqvist et al. (2020), Wadekar et al. (2019), Wan et al. (2021), Wang, Chen, et al. (2021), Wang et al. (2022), Wang, Zhang, Wang and Cao (2021), Xi et al. (2021), Xia et al. (2021), Xiao et al. (2019), Xie et al. (2020), Yang et al. (2021, 2019), Yen et al. (2019), Yin et al. (2020), Yu et al. (2021), Zhang, Li, et al. (2021), Zhang, Wang, et al. (2021), Zhang et al. (2019), Zhao, Niu, et al. (2021), Zhou et al. (2020), Zhu et al. (2020)
BlueGene/L (BGL) (Oliner & Stearley, 2007)	High-perf. comp.	✓	–	Failures	Catillo et al. (2022), Chen, and Xiao and Jin (2021), Du et al. (2017, 2021), Farzad (2020), Farzad and Gulliver (2019, 2021), Gu et al. (2021), Guo, Lin, et al. (2021), Guo, Yuan, and Wu (2021), Han and Yuan (2021), Hashemi and Mäntylä (2021), Hirakawa et al. (2020), Huang et al. (2020), Le and Zhang (2021), Li et al. (2020), Liu et al. (2021), Lv et al. (2021), Meng et al. (2019), Nedelkoski et al. (2020), Studiawan et al. (2020), Sun et al. (2020), Wan et al. (2021), Wang, Chen, et al. (2021), Wang et al. (2022), Wang, Zhang, Wang and Cao (2021), Wittkopp et al. (2021), Xi et al. (2021), Xia et al. (2021), Yang et al. (2021, 2019), Zhang, Li, et al. (2021), Zhang, Wang, et al. (2021)
Thunderbird (Oliner & Stearley, 2007)	High-perf. comp.	✓	–	Failures	Farzad (2020), Farzad and Gulliver (2019, 2021), Guo, Lin, et al. (2021), Guo, Wu, et al. (2021), Guo, Yuan, and Wu (2021), Han and Yuan (2021), Le and Zhang (2021), Nedelkoski et al. (2020), Sun et al. (2020), Wang et al. (2018), Wittkopp et al. (2021), Yin et al. (2020), Zhang, Li, et al. (2021)
OpenStack (Du et al., 2017)	Virtual machines	✓	✓	Failures	Baril et al. (2020), Du et al. (2017), Farzad (2020), Farzad and Gulliver (2019, 2021), Hashemi and Mäntylä (2021), Huang et al. (2020), Ott et al. (2021), Qian et al. (2020), Wibisono and Kistijantoro (2019), Zhao, Niu, et al. (2021), Zhou et al. (2020), Zhu et al. (2020)
Hadoop (Lin, Zhang, Lou, Zhang, & Chen, 2016)	High-perf. comp.	✓	✓	Failures	Catillo et al. (2022), Chen, Zhang, et al. (2020), Hashemi and Mäntylä (2021), Studiawan et al. (2020)
Spirit (Oliner & Stearley, 2007)	High-perf. comp.	✓	–	Failures	Le and Zhang (2021), Nedelkoski et al. (2020), Wittkopp et al. (2021)
Digital Corpora (Garfinkel, Farrell, Roussev, & Dinolt, 2009)	OS logs	–	–	Failures	Studiawan and Sohel (2021), Studiawan et al. (2020)
DFRWS 2009 (Eoghan & Golden, 2022)	OS logs	–	–	Exfiltration	Studiawan and Sohel (2021), Studiawan et al. (2020)
Honeynet 2011 (Arcas, Gonzales, & Cheng, 2022)	OS logs	–	–	Intrusions	Studiawan and Sohel (2021), Studiawan et al. (2020)
Windows (He, Zhu, He, & Lyu, 2020)	OS logs	–	–	–	Studiawan and Sohel (2021), Studiawan et al. (2020)
Linux Security Logs (Chuvakin, 2010)	OS logs	–	–	Intrusions	Studiawan and Sohel (2021)

(continued on next page)

Table 4 (*continued*).

Data set	Use-case	Labels	Sessions	Anomaly sources	Used in evaluation
Honeynet 2010 (Marty, Chuvakin, & Tricaud, 2022)	OS logs	–	–	Intrusions	Studiawan et al. (2020)
Android (He et al., 2020)	Mobile OS logs	–	–	–	Li et al. (2020)
HPC RAS (Zheng et al., 2011)	High-perf. comp.	–	✓	Failures	Nedelkoski et al. (2020)
Spark (He et al., 2020)	High-perf. comp.	–	–	Failures	Studiawan et al. (2020)
Zookeeper (He et al., 2020)	High-perf. comp.	–	–	Failures	Studiawan et al. (2020)

4.5.2. Evaluation metrics

Quantitative evaluation (ER-2) of anomaly detection approaches typically revolves around counting the numbers of correctly detected anomalous samples as true positives (TP), incorrectly detected non-anomalous samples as false positives (FP), incorrectly undetected anomalous samples as false negatives (FN), and correctly undetected non-anomalous samples as true negatives (TN). In the most basic setting where events are labeled individually and samples represent single events (e.g., as in the BGL data set), it is relatively straightforward to evaluate detected events with binary classification (Farzad & Gulliver, 2019, 2021). Some of the reviewed papers additionally consider a multi-class classification problem for data sets where different types of failures have distinct labels by computing the averages of evaluation metrics over all classes (Sun et al., 2020) or plotting confusion matrices (Decker et al., 2020).

Given that log events are sometimes aggregated with diverse methods prior to detection, it stands to reason that there are different ways to determine whether a sample is anomalous or not, and whether it counts as a correct detection or not. For example, aggregation of logs in windows could require to count detected events as true positives as long as they are close enough to the actual anomaly in the event sequence (Baril et al., 2020; Syngal et al., 2021). Since a majority of the reviewed papers rely on the HDFS data set where labels are only available for whole event sessions rather than single events, the most common method to compute aforementioned metrics relies on counting of in-/correctly identified non-/anomalous sessions (Chen, and Xiao & Jin, 2021; Du et al., 2021; Huang et al., 2020; Li & Li, 2020; Wang, Chen, et al., 2021).

Regardless of how the positive and negative samples are counted, almost all authors eventually evaluate their approaches using the well-known metrics precision ($P = \frac{TP}{TP+FP}$), recall or true positive rate ($R = TPR = \frac{TP}{TP+FN}$), false positive rate ($FPR = \frac{FP}{FP+TN}$) and F1-score ($F1 = \frac{2 \cdot P \cdot R}{P+R}$). Less common evaluation metrics are the accuracy ($ACC = \frac{TP+TN}{TP+TN+FP+FN}$) used in 15 publications as well as the area under curve which is computed for precision–recall-curves (Chen, Zhang, et al., 2020) and receiver operator characteristic (ROC) curves (Hirakawa et al., 2020; Wang et al., 2018). Other metrics that are more specific to deep learning applications are the number of model parameters (Guo, Wu, et al., 2021; Wang et al., 2022) and time to train models or run the detection (ER-3) (Cheansunan & Phunchongharn, 2019; Decker et al., 2020; Gu et al., 2021; Liu et al., 2021; Qian et al., 2020; Xie et al., 2020). Some authors also assess characteristics of their approaches that go beyond standard anomaly detection evaluations, for example, whether training on combinations of multiple data sets improves the overall performance of classification (Hashemi & Mäntylä, 2021) or whether their approaches are robust against changes of log patterns over time (Hashemi & Mäntylä, 2021; Huang et al., 2020; Zhang et al., 2019).

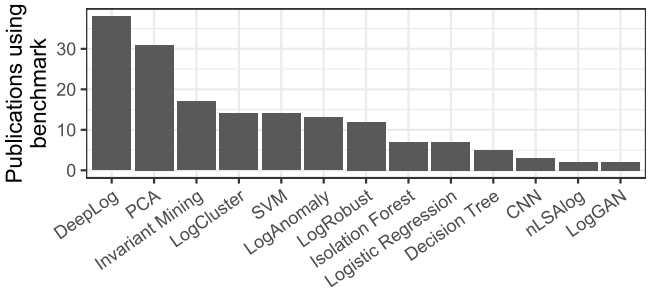


Fig. 5. Overview of common benchmark approaches used in evaluations.

4.5.3. Benchmark approaches

Most publications compare the evaluation metrics stated in the previous section with benchmark approaches to show their improvements over the state-of-the-art (ER-4). Fig. 5 shows the most commonly used benchmarks in the reviewed publications. Note that we only considered approaches that appear in at least two different publications for this visualization.

As visible in the figure, **DeepLog** (Du et al., 2017) is the most commonly used benchmark with appearances in 38 out of the 62 reviewed publications. DeepLog relies on LSTM RNNs to predict upcoming log events in sequences and thereby disclose observed events as anomalies if they are expected to occur with low probabilities. The popularity of DeepLog for comparisons can be explained by the facts that DeepLog was the first to detect sequential anomalies in log data using deep learning (cf. Section 4.1.2) and that open-source re-implementations are available online.

The second most commonly used benchmark leverages principal component analysis (**PCA**) and relies on event counts rather than sequences. In particular, Xu et al. (2009a) create message count vectors for event type identifiers and use PCA to transform them into subspaces where anomalies appear as outliers that have a high distance to all other samples. Lou, Fu, Yang, Xu, and Li (2010) also generate message count vectors but use **Invariant Mining** to discover linear relationships between log events that represent execution workflows. Event sequences that violate previously identified invariants are declared as anomalies.

LogCluster (Lin et al., 2016) generates vectors for log sequences and then clusters them using a similarity metric. The approach is mainly designed for log filtering but can also be applied for detection of unusual log patterns. Support vector machines (**SVM**) (Liang, Zhang, Xiong, & Sahoo, 2007) are yet another method relying on event count vectors. However, other than PCA and invariant mining, SVM typically operate in supervised manner. To alleviate this issue and enable application in semi- or unsupervised cases, authors therefore resort to one-class SVM (Schölkopf, Platt, Shawe-Taylor, Smola, & Williamson, 2001) or Support Vector Data Description (SVDD) (Tax & Duin, 2004).

Similar to DeepLog and contrary to aforementioned conventional machine learning approaches, **LogAnomaly** (Meng et al., 2019) leverages LSTM RNNs to analyze log event sequences. To this end, they propose the so-called template2Vec embedding method to extract semantic vectors from the tokens that make up the events. **LogRobust** (Zhang et al., 2019) also makes use of semantic vectors but is specifically designed to handle unknown log events that appear as part of software evolution.

Isolation Forest (Liu, Ting, & Zhou, 2008) is an anomaly detection technique where the analyzed data is recursively split until a single data instance is isolated from all other points; thereby, anomalous points are identified as they are expected to require fewer splits until their isolation. **Logistic Regression** (Bodik, Goldszmidt, Fox, Woodard, & Andersen, 2010) is a classifier for numeric input data that works best in linear classification cases (Studiawan & Sohel, 2021). **Decision Tree** (Chen, Zheng, Lloyd, Jordan, & Brewer, 2004) on the other hand is a supervised classification method where instances traverse a binary search tree. Thereby, each internal node splits the data by a specific predicate and each leaf of the tree determines the class of the instances.

Another benchmark that relies on deep learning are **CNN**. In particular, the approach by Lu et al. (2018) semantically encodes sequences of event identifiers and embeds them into a matrix for convolution. Finally, **nLSAlog** (Yang et al., 2019) leverages LSTM RNNs and is thus similar to DeepLog. Out of all reviewed publications, only eight do not involve any benchmark approach for comparison.

4.5.4. Reproducibility

Authors of scientific publications should pursue to enable reproducibility of their presented results for many reasons, including the possibility for others to validate the correctness of the approach, to extend the algorithms with additional features, to carry out their own experiments on other data sets, and to use the approach as benchmarks in new publications. We consider an approach reproducible when both the data used in the evaluation (ER-5) as well as the original source code (ER-6) are publicly available.

As outlined in Section 4.5.1, a majority of the reviewed publications carry out their evaluations on the same few data sets that are publicly available. Some authors also evaluate their approaches on private data sets that are synthetically generated in testbeds (Baril et al., 2020; Li & Li, 2020; Sundqvist et al., 2020; Wang, Chen, et al., 2021; Wibisono & Kistijantoro, 2019; Yang & Agrawal, 2016; Zhang, Dai, et al., 2021), collected from academic institutions (Otomo et al., 2019; Yang et al., 2021), or obtained from industrial real-world applications (Catillo et al., 2022; Chen, Zhang, et al., 2020; Decker et al., 2020; Syngal et al., 2021; Wittkopp et al., 2021; Yang et al., 2021; Zhang et al., 2019). Overall, 55 out of the 62 reviewed publications involve evaluations on at least one of the publicly available data sets from Table 4.

While it is relatively common to evaluate approaches on public data sets, there are unfortunately only few authors that publish implementations of their approaches alongside the papers. During our review we were only able to find the original source code of presented approaches from 8 publications. However, we also point out that there exist some re-implementations of scientific approaches in the deep-loglizer toolbox provided by Chen, Liu, et al. (2021). We mark approaches where implementations by the original authors exist with (YES), re-implementations by other authors as (RE), and all others as (NO) in Section 4. We encourage authors to publish their code to improve the reproducibility of their results and hope to see more open-source implementations in the future.

5. Discussion

The previous sections presented the results of our survey in detail. In the following we summarize these results, discuss open issues in the research area on log-based anomaly detection using deep learning, and propose ideas for future research in course of answering our research questions from Section 1.

***RQ1:** What are the main challenges of log-based anomaly detection with deep learning?* When carrying out our systematic literature review we assessed whether and to what degree the current state-of-the-art addresses the research challenges enumerated in Section 2.2. It turns out that data instability, i.e., the appearance of previously unknown events, is one of the main issues addressed by the reviewed approaches. The key idea to resolving this problem is currently to represent logs as semantic vectors so that new or changed events can still be compared to known events by measuring their similarities (Catillo et al., 2022; Han & Yuan, 2021; Le & Zhang, 2021; Li et al., 2020; Lv et al., 2021; Ott et al., 2021; Syngal et al., 2021; Xi et al., 2021; Yang et al., 2021; Zhang, Li, et al., 2021; Zhang et al., 2019). There are many techniques for generating numeric vectors to represent log events (cf. Section 4.3.4) and thus resolve the issue of feeding unstructured and textual input data to neural networks. Imbalanced data sets are another challenge that is specifically addressed by some approaches. In particular, authors suggest to use sampling techniques as well as context-aware embedding methods as possible solutions (Farzad, 2020; Li & Li, 2020; Sun et al., 2020; Wang, Chen, et al., 2021; Xia et al., 2021).

Some approaches are specifically designed to enable applicability in scenarios that demand efficient and lightweight algorithms, e.g., deployment on edge devices. This is achieved by leveraging low-dimensional vector representations as well as convolutional neural networks that are more efficient than recurrent neural networks (Cheansunan & Phunchongharn, 2019; Guo, Wu, et al., 2021; Wang et al., 2022). Similarly, some approaches support log stream processing and enable adaptive learning (i.e., dynamically changing the baselines for anomaly detection) by incrementally re-training the models with manually identified and labeled false positive samples (Yen et al., 2019). It must be noted that there is currently no solution how to automatically determine that re-training is required without label information or manual intervention. The challenge of interleaving logs is generally solved by leveraging session identifiers directly from parsed log data; however, there are also approaches that evade the need for sequences altogether, e.g., by relying on sentiment analysis (Zhang, Dai, et al., 2021).

A way to address the challenge that only few labeled data is available is provided by transfer learning, where models are trained on one system and tested on another (Chen, Zhang, et al., 2020; Guo, Lin, et al., 2021). The main idea is that the log event patterns learned by the neural networks are similar across different domains and that already seen anomalies can be recognized and classified. Guo, Wu, et al. (2021) are the only authors to consider federated learning, where learning takes place in a distributed manner across multiple systems. Hashemi and Mäntylä (2021) also go into this direction as they combine multiple data sets to evaluate whether this affects the performance of their model. We believe that federated learning could be an interesting topic for future publications as there exist many real-world scenarios where log data is monitored in distributed machines but orchestration of deployed detectors takes place centrally (Preuveneers et al., 2018).

The challenge of facing diverse artifacts of anomalies is only partially addressed since the vast majority of approaches focus on sequences and frequencies of log events, but only few consider event parameters or inter-arrival times for detection. We recommend to also consider techniques that address other patterns that appear in normal system behavior and may be useful to detect specific anomalies, such as changes of parameter value correlations, periodic behavior, statistical distributions, etc. Moreover, we observed that the explainability of the proposed deep learning models is relatively low, i.e., it is non-trivial to understand the criteria for classifications and thus detection of model bias as well as interpretation of false positives and false negatives is generally difficult. This hinders root cause analysis of detected anomalies and produces an overhead for system operators.

Specifically in security-critical systems (e.g., intrusion detection) it is vital to understand the functioning – and thus the limits – of deployed anomaly detectors. We would therefore recommend to direct future research in log-based anomaly detection towards explainable artificial intelligence (Arrieta et al., 2020).

***RQ2**: What state-of-the-art deep learning algorithms are typically applied?* Our review shows that diverse types of deep learning algorithms are used in scientific publications and that it is common to combine several approaches. Thereby, RNNs are clearly the most applied models, because they are a natural choice for capturing sequential patterns in log data. CNNs are used as an efficient alternative to RNNs as they are also able to pick up event dependencies. On the other hand, Autoencoders and Transformers are frequently applied as they support unsupervised learning. While GANs, MLPs, GNNs, and EGNNs are only used by few approaches, they have beneficial properties (cf. Section 4.2.1) that make these models worth considering. Similarly, we are convinced that other deep learning architectures that are not explored in the reviewed literature could yield interesting insights, e.g., deep belief networks or deep reinforcement learning (Sarker, 2021a, 2021b).

We believe that the application of specific techniques is mostly motivated by the log data to be analyzed and anomalies to be detected. The fact that all of the commonly used log data sets involve anomalies that manifest as sequentially occurring events (cf. Section 4.5.1) thus explains the tendency towards RNNs. However, as anomalies could also manifest as point or contextual anomalies (cf. Section 2.1.3) we recommend to consider alternative log data sets with different types of anomalies and to develop approaches for these cases. For example, in our earlier works (Landauer et al., 2022; Landauer, Skopik, Wurzenberger, Hotwagner, & Rauber, 2020) we published log data sets where anomalies affect combinations, compositions, and distributions of event parameter values in addition to frequencies and sequences of log events.

***RQ3**: How is log data pre-processed to be ingested by deep learning models?* Our survey shows that there are three main ways to approach feature extraction from raw log data. First, by tokenizing log messages, which is a simple method that does not require any parsers but lacks semantic interpretation of the tokens. Second, by parsing the messages and extracting information from collections of log events, such as sequences, counts, or statistics. Third, by extracting parameters including the time stamps from parsed log events. There are a multitude of methods to represent these features as numeric vectors to be ingested by deep learning models.

While traditional machine learning methods such as SVM or PCA work best with event count vectors, most approaches leveraging deep learning neural networks use semantic vectors to yield the best results (Wang, Zhang, Wang & Cao, 2021). Thereby, the tokens that make up the log messages are represented as numeric vectors and considered in the context of their sequence of event occurrences. Most approaches employ these sequential features, while frequencies, one-hot encoded data, and embedding layers are used less often or only as a contributing feature.

***RQ4**: What types of anomalies are detected and how are they identified as such?* Almost all reviewed approaches focus on sequential anomalies that either manifest in the sequences of events, the sequences of tokens within events, or a combination of both. Only few approaches make use of event counts or detect single log lines as outliers without their context of occurrence. The detection technique is generally driven by the output of the neural networks. While binary or multi-class classifications are directly used to report anomalies, all numeric outputs such as anomaly scores or reconstruction errors are compared against pre-defined thresholds and probability distributions of log events are used to check whether the actual events is within the top candidates. Determining these thresholds is usually carried out empirically for a particular log file.

***RQ5**: How are the proposed models evaluated?* Our survey on commonly used log data sets from Section 4.5.1 shows that there are only four data sets that are used by a majority of the approaches: HDFS, BGL, Thunderbird, and OpenStack. All these data sets are collected from scenarios involving high-performance computing and virtual machines that are affected by randomly occurring failures. It is obvious that the availability of anomaly labels make these data sets particularly attractive for scientific evaluations. As mentioned before, we argue that a larger and more diverse set of input data sets would be beneficial to evaluate whether the proposed approaches are capable of detecting anomalous artifacts other than unusual sequences. In particular, the consequences of cyber attacks rather than failures could result in log artifacts that are suitable for detection and an appropriate use-case for anomaly detection.

We manually analyzed the HDFS data set as it is the most popular of all and found that it is far from challenging to achieve competitive detection rates. The reason for this is that many of the anomalous event sequences are trivial to identify as they involve event types that never occur in the training data or involve fewer elements than the shortest normal sequences. Using these two heuristics we are able to achieve $F1 = 90.41\%$, $ACC = 99.48\%$, $P = 98.37\%$, $R = 83.65\%$, $FPR = 0.04\%$ on the test data. Moreover, using these heuristics in combination with the simple Stide algorithm (Forrest, Hofmeyr, Somayaji, & Longstaff, 1996) that moves a sliding window of a given length over the data and looks for sub-sequences that have not been seen before in the training data further improves the evaluation metrics to $F1 = 95.14\%$, $ACC = 99.71\%$, $P = 95.27\%$, $R = 95.01\%$, $FPR = 0.14\%$ when using a window size of 2. Omitting sequential information altogether and only leveraging similarities of event count vectors further pushes these evaluation metrics to $F1 = 98.86\%$, $ACC = 99.93\%$, $P = 97.81\%$, $R = 99.92\%$, $FPR = 00.07\%$. For comparison, DeepLog only yields detection scores of $F1 = 95.72\%$, $ACC = 99.75\%$, $P = 95.12\%$, $R = 96.32\%$, $FPR = 0.15\%$ (Du et al., 2017). We provide the code for our experiments in a reproducible form as open-source implementations (in separate repositories for Stide[10] and similarity-based event count vector clustering[11]). It is not clear to us why such approaches were not used as benchmarks in any of the reviewed publications: They only take a fraction of the time for training and processing the test data in comparison to deep (and also most conventional) learning models, and additionally have a much better explainability than neural networks. Similar conclusions have been drawn for the case of log event prediction using the HDFS data set (Mäntylä, Varela, & Hashemi, 2022). We argue that the fact that such simple algorithms achieve competitive detection rates to deep learning models further urges authors to consider additional data sets where more diverse anomalous artifacts are present and the benefits of their approaches such as robustness against data instability become apparent.

Another issue that we noticed is that evaluations are usually carried out on the basis of anomalous sequences, i.e., the whole sequence is considered normal or anomalous rather than its elements (Du et al., 2017). However, parts of long sequences may actually represent normal system behavior while only a few elements should be considered anomalies. It would be interesting to evaluate whether detection approaches are able to pinpoint exactly which parts of sequences are anomalous, which would also be practical for manual investigations of reported anomalies by system operators. Obviously this requires that data sets are labeled on the granularity of single events rather than sessions (Mäntylä et al., 2022).

Finally, we note that almost all evaluations rely on metrics such as the F-score (cf. Section 4.5.2) that are known to not accurately depict the classification or detection performance when data sets are highly imbalanced. To avoid misinterpretations of evaluation results, it is recommended to also compute metrics that are more robust against class imbalance, such as the specificity or true negative rate (Le & Zhang, 2022).

[10] https://github.com/ait-aecid/stide

[11] https://github.com/ait-aecid/count-vector-clustering

***RQ6**: To what extent do the approaches rely on labeled data and support incremental learning?* Log-based anomaly detection is most often applied in use-cases that aim to disclose unexpected system behavior such as failures or cyber attacks. Since these artifacts are not known beforehand and thus no labels can exist, un- or semi-supervised approaches are generally more widely applicable and therefore preferable (Chandola et al., 2009). Since semi-supervised learning can be achieved with most neural network architectures including RNNs, but only specific deep learning models support fully unsupervised operation, we find 28 semi-supervised approaches in our reviewed literature as opposed to only 8 out of 62 approaches that are unsupervised. We did not expect to see the relatively large amount of 26 supervised approaches that require at least partially labeled anomalies for training. Moreover, with 54 out of 62 a vast majority of approaches only support offline training. This includes most supervised models and also all other approaches that do not intend to dynamically and automatically update the trained models over time. Only 8 of the reviewed approaches enable continuous model adjustments through re-training or EGNN model architectures (Decker et al., 2020).

***RQ7**: How reproducible are the presented results in terms of availability of source code and used data?* As pointed out in Section 4.5.4, a majority of the reviewed approaches make use of publicly available data sets to evaluate their approaches and only 7 publications rely on private data sets. The reproducibility of the presented results is thus relatively high, assuming that readers are willing to re-implement the approaches based on the descriptions from the papers from scratch. We could only find 9 publicly available source codes of approaches published by the original authors as well as 4 re-implementations, indicating a low reproducibility overall. We encourage authors to publish reproducible experiments in the future to also enable large-scale quantitative comparisons in surveys.

Recommendations. Based on the aforementioned answers to our research questions and the identified issues, we propose the following research agenda: First of all, adequate and diverse log data sets with state-of-the-art benchmarks are needed to ensure applicability of deep learning in generic anomaly detection use-cases and demonstrate their superiority over simple or conventional detection methods. Second, low explainability of detection results is a primary concern that permeates the entire research field and needs to be addressed appropriately. Resolving these two issues largely improves the comprehensibility and reliability of proposed methods and facilitates the development of novel deep learning detection algorithms. With this research agenda in mind, we summarize our recommendations for future research as follows.

- Create or identify new log data sets that specifically involve sequential anomalies and are less affected by other types of anomalies when evaluating approaches that ingest log data as sequences.
- Work on methods that improve the explainability of proposed approaches for anomaly detection using deep learning, for example, by the extraction of specific detection rules from the models or by determining the main features responsible for the detection of specific instances and augmenting detection results with that information.
- Consider log artifacts other than event sequences for anomaly detection or use them as additional input to deep learning models.
- Propose novel anomaly detection methods or deep learning architectures that resolve common challenges for practical applications, specifically regarding incremental and stream processing or log data, adaptive learning, as well as efficient and low-resource training.
- Consider pinpointing anomalies within sequences rather than detecting whole sequences as anomalous.
- Allow researchers to reproduce and extend presented results by publishing developed code as open-source and used log data sets on data sharing repositories.

6. Conclusion

This paper presents a survey of 62 scientific approaches that pursue the detection of anomalous events or processes in system log data using deep learning. The survey shows that diverse model architectures are suitable for this purpose, including models for sequential input data such as recurrent or convolutional neural networks, language-based models such as transformers, as well as unsupervised models such as Autoencoders or generative adversarial networks. Similarly, there are different features used for training and subsequently for detection, such as sequences and counts of events or tokens as well as parameter values or statistics derived from the events. To enable processing of these features as input to neural networks it is necessary to encode them as numeric vectors, for example, through semantic vectorization or one-hot encoding. Anomalies are then detected either directly through classification or by deriving some kind of anomaly score from the network that allows to discern normal from anomalous system behavior. The survey shows that there are open challenges that are not sufficiently resolved by existing approaches, including detection techniques that go beyond sequential anomalies, low explainability of trained models and classification results, lack of representative evaluation data sets containing diverse attack artifacts, and a low reproducibility.

CRediT authorship contribution statement

Max Landauer: Conceptualization, Methodology, Validation, Writing – original draft, Visualization. **Sebastian Onder:** Investigation, Resources, Data curation, Formal analysis. **Florian Skopik:** Supervision, Funding acquisition, Writing – review & editing. **Markus Wurzenberger:** Supervision, Funding acquisition, Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Max Landauer reports financial support was provided by Austrian Research Promotion Agency (FFG).

Max Landauer reports financial support was provided by European Defence Fund (EDF).

Data availability

No data was used for the research described in the article

Acknowledgments

This work was partly funded by the European Defence Fund (EDF) projects AInception (101103385) and PANDORA (SI2.835928), and the FFG project DECEPT (873980).

References

Arcas, G., Gonzales, H., & Cheng, J. (2022). The honeynet project 2011 challenge 7 – forensic analysis of a compromised server. https://www.honeynet.org/challenges/forensic-challenge-7-analysis-of-a-compromised-server/. (Accessed 13 May 2022).

Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., et al. (2020). Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, *58*, 82–115.

Bao, L., Li, Q., Lu, P., Lu, J., Ruan, T., & Zhang, K. (2018). Execution anomaly detection in large-scale systems through console log analysis. *Journal of Systems and Software*, *143*, 172–186.

Baril, X., Coustié, O., Mothe, J., & Teste, O. (2020). Application performance anomaly detection with LSTM on temporal irregularities in logs. In *Proceedings of the 29th ACM international conference on information & knowledge management* (pp. 1961–1964).

Bhanage, D. A., Pawar, A. V., & Kotecha, K. (2021). IT infrastructure anomaly detection and failure handling: A systematic literature review focusing on datasets, log preprocessing, machine & deep learning approaches and automated tool. *IEEE Access*.

Bodik, P., Goldszmidt, M., Fox, A., Woodard, D. B., & Andersen, H. (2010). Fingerprinting the datacenter: Automated classification of performance crises. In *Proceedings of the 5th european conference on computer systems* (pp. 111–124).

Bursic, S., Cuculo, V., & D'Amelio, A. (2019). Anomaly detection from log files using unsupervised deep learning. In *International symposium on formal methods* (pp. 200–207). Springer.

Catillo, M., Pecchia, A., & Villano, U. (2022). AutoLog: Anomaly detection by deep autoencoding of system logs. *Expert Systems with Applications*, *191*, Article 116263.

Chalapathy, R., & Chawla, S. (2019). Deep learning for anomaly detection: A survey. arXiv preprint arXiv:1901.03407.

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, *41*(3), 1–58.

Cheansunan, P., & Phunchongharn, P. (2019). Detecting anomalous events on distributed systems using convolutional neural networks. In *2019 IEEE 10th international conference on awareness science and technology* (pp. 1–5). IEEE.

Chen, Z., Liu, J., Gu, W., Su, Y., & Lyu, M. R. (2021). Experience report: Deep learning-based system log analysis for anomaly detection. arXiv preprint arXiv:2107.05908.

Chen, H., Xiao, R., & Jin, S. (2021). Unsupervised anomaly detection based on system logs. In *Proceedings of the 33rd international conference on software engineering & knowledge engineering* (pp. 92–97).

Chen, R., Zhang, S., Li, D., Zhang, Y., Guo, F., Meng, W., et al. (2020). Logtransfer: Cross-system log anomaly detection for software systems with transfer learning. In *2020 IEEE 31st international symposium on software reliability engineering* (pp. 37–47). IEEE.

Chen, M., Zheng, A. X., Lloyd, J., Jordan, M. I., & Brewer, E. (2004). Failure diagnosis using decision trees. In *International conference on autonomic computing, 2004. Proceedings* (pp. 36–43). IEEE.

Chuvakin, A. (2010). *Public security log sharing site*. https://log-sharing.dreamhosters.com/. (Accessed 18 October 2021).

Cui, Y., Ahmad, S., & Hawkins, J. (2016). Continuous online sequence learning with an unsupervised neural network model. *Neural Computation*, *28*(11), 2474–2504.

Decker, L., Leite, D., Viola, F., & Bonacorsi, D. (2020). Comparison of evolving granular classifiers applied to anomaly detection for predictive maintenance in computing centers. In *2020 IEEE conference on evolving and adaptive intelligent systems* (pp. 1–8). IEEE.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

Du, M., & Li, F. (2016). Spell: Streaming parsing of system event logs. In *2016 IEEE 16th international conference on data mining* (pp. 859–864). IEEE.

Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security* (pp. 1285–1298).

Du, Q., Zhao, L., Xu, J., Han, Y., & Zhang, S. (2021). Log-based anomaly detection with multi-head scaled dot-product attention mechanism. In *International conference on database and expert systems applications* (pp. 335–347). Springer.

Eoghan, C., & Golden, G. R. I. (2022). DFRWS 2009 forensics challenge. http://old.dfrws.org/2009/challenge/index.shtml. (Accessed 13 May 2022).

Farzad, A. (2020). Log message anomaly detection with oversampling. *International Journal of Artificial Intelligence and Applications (IJAIA)*, *11*(4).

Farzad, A., & Gulliver, T. A. (2019). Log message anomaly detection and classification using auto-B/LSTM and auto-GRU. arXiv preprint arXiv:1911.08744.

Farzad, A., & Gulliver, T. A. (2021). Two class pruned log message anomaly detection. *SN Computer Science*, *2*(5), 1–18.

Fischer, D. A., Goel, K., Andrews, R., van Dun, C. G. J., Wynn, M. T., & Röglinger, M. (2020). Enhancing event log quality: Detecting and quantifying timestamp imperfections. In *Business process management: 18th international conference* (pp. 309–326). Springer.

Forrest, S., Hofmeyr, S. A., Somayaji, A., & Longstaff, T. A. (1996). A sense of self for unix processes. In *Proceedings 1996 IEEE symposium on security and privacy* (pp. 120–128). IEEE.

Garfinkel, S., Farrell, P., Roussev, V., & Dinolt, G. (2009). Bringing science to digital forensics with standardized forensic corpora. *Digital Investigation*, *6*, S2–S11.

Gu, S., Chu, Y., Zhang, W., Liu, P., Yin, Q., & Li, Q. (2021). Research on system log anomaly detection combining two-way slice GRU and GA-attention mechanism. In *2021 4th international conference on artificial intelligence and big data* (pp. 577–583). IEEE.

Guo, H., Lin, X., Yang, J., Zhuang, Y., Bai, J., Zhang, B., et al. (2021). TransLog: A unified transformer-based framework for log anomaly detection. arXiv preprint arXiv:2201.00016.

Guo, Y., Wu, Y., Zhu, Y., Yang, B., & Han, C. (2021). Anomaly detection using distributed log data: A lightweight federated learning approach. In *2021 international joint conference on neural networks* (pp. 1–8). IEEE.

Guo, H., Yuan, S., & Wu, X. (2021). Logbert: Log anomaly detection via bert. In *2021 international joint conference on neural networks* (pp. 1–8). IEEE.

Hadsell, R., Rao, D., Rusu, A. A., & Pascanu, R. (2020). Embracing change: Continual learning in deep neural networks. *Trends in Cognitive Sciences*, *24*(12), 1028–1040.

Han, X., & Yuan, S. (2021). Unsupervised cross-system log anomaly detection via domain adaptation. In *Proceedings of the 30th ACM international conference on information & knowledge management* (pp. 3068–3072).

Hashemi, S., & Mäntylä, M. (2021). OneLog: Towards end-to-end training in software log anomaly detection. arXiv preprint arXiv:2104.07324.

He, S., Zhu, J., He, P., & Lyu, M. R. (2016). Experience report: System log analysis for anomaly detection. In *2016 IEEE 27th international symposium on software reliability engineering* (pp. 207–218). IEEE.

He, S., Zhu, J., He, P., & Lyu, M. R. (2020). Loghub: A large collection of system log datasets towards automated log analytics. arXiv preprint arXiv:2008.06448.

He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE international conference on web services* (pp. 33–40). IEEE.

Hirakawa, R., Tominaga, K., & Nakatoh, Y. (2020). Software log anomaly detection through one class clustering of transformer encoder representation. In *International conference on human-computer interaction* (pp. 655–661). Springer.

Huang, S., Liu, Y., Fung, C., He, R., Zhao, Y., Yang, H., et al. (2020). Hitanomaly: Hierarchical transformers for anomaly detection in system log. *IEEE Transactions on Network and Service Management*, *17*(4), 2064–2076.

Kruegel, C., & Vigna, G. (2003). Anomaly detection of web-based attacks. In *Proceedings of the 10th ACM conference on computer and communications security* (pp. 251–261).

Kwon, D., Kim, H., Kim, J., Suh, S. C., Kim, I., & Kim, K. J. (2019). A survey of deep learning-based network anomaly detection. *Cluster Computing*, *22*(1), 949–961.

Landauer, M., Skopik, F., Frank, M., Hotwagner, W., Wurzenberger, M., & Rauber, A. (2022). Maintainable log datasets for evaluation of intrusion detection systems. arXiv preprint arXiv:2203.08580.

Landauer, M., Skopik, F., Wurzenberger, M., Hotwagner, W., & Rauber, A. (2020). Have it your way: Generating customized log datasets with a model-driven simulation testbed. *IEEE Transactions on Reliability*, *70*(1), 402–415.

Landauer, M., Skopik, F., Wurzenberger, M., & Rauber, A. (2020). System log clustering approaches for cyber security applications: A survey. *Computers & Security*, *92*, Article 101739.

Landauer, M., Wurzenberger, M., Skopik, F., Settanni, G., & Filzmoser, P. (2018). Dynamic log file analysis: An unsupervised cluster evolution approach for anomaly detection. *Computers & Security*, *79*, 94–116.

Le, V. H., & Zhang, H. (2021). Log-based anomaly detection without log parsing. In *2021 36th IEEE/ACM international conference on automated software engineering* (pp. 492–504). IEEE.

Le, V. H., & Zhang, H. (2022). Log-based anomaly detection with deep learning: How far are we? In *Proceedings of the 44th international conference on software engineering* (pp. 1356–1367).

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436–444.

Li, X., Chen, P., Jing, L., He, Z., & Yu, G. (2020). Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults. In *2020 IEEE 31st international symposium on software reliability engineering* (pp. 92–103). IEEE.

Li, H., & Li, Y. (2020). Logspy: System log anomaly detection for distributed systems. In *2020 international conference on artificial intelligence and computer engineering* (pp. 347–352). IEEE.

Liang, Y., Zhang, Y., Xiong, H., & Sahoo, R. (2007). Failure prediction in ibm bluegene/l event logs. In *Seventh IEEE international conference on data mining* (pp. 583–588). IEEE.

Liao, H. J., Lin, C. H. R., Lin, Y. C., & Tung, K. Y. (2013). Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications*, *36*(1), 16–24.

Lin, Q., Zhang, H., Lou, J. G., Zhang, Y., & Chen, X. (2016). Log clustering based problem identification for online service systems. In *Proceedings of the 38th international conference on software engineering companion* (pp. 102–111).

Liu, X., Liu, W., Di, X., Li, J., Cai, B., Ren, W., et al. (2021). LogNADS: Network anomaly detection scheme based on log semantics representation. *Future Generation Computer Systems*, *124*, 390–405.

Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. In *2008 eighth ieee international conference on data mining* (pp. 413–422). IEEE.

Lou, J. G., Fu, Q., Yang, S., Xu, Y., & Li, J. (2010). Mining invariants from console logs for system problem detection. In *2010 USENIX annual technical conference*.

Lu, S., Wei, X., Li, Y., & Wang, L. (2018). Detecting anomaly in big data system logs using convolutional neural network. In *2018 IEEE 16th intl conf on dependable, autonomic and secure computing, 16th intl conf on pervasive intelligence and computing, 4th intl conf on big data intelligence and computing and cyber science and technology congress (DASC/PiCom/DataCom/CyberSciTech)* (pp. 151–158). IEEE.

Lv, D., Luktarhan, N., & Chen, Y. (2021). ConAnomaly: Content-based anomaly detection for system logs. *Sensors*, *21*(18), 6125.

Manning, C., Raghavan, P., & Schütze, H. (2010). Introduction to information retrieval. *Natural Language Engineering*, *16*(1), 100–103.

Mäntylä, M., Varela, M., & Hashemi, S. (2022). Pinpointing anomaly events in logs from stability testing–N-grams vs. deep-learning. In *2022 IEEE international conference on software testing, verification and validation workshops* (pp. 285–292). IEEE.

Marty, R., Chuvakin, A., & Tricaud, S. (2022). The honeynet project 2010 challenge 5 – log mysteries. https://www.honeynet.org/challenges/forensic-challenge-7-analysis-of-a-compromised-server/. (Accessed 13 May 2022).

Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., et al. (2019). LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *IJCAI (vol. 19), no. 7* (pp. 4739–4745).

Mi, H., Wang, H., Zhou, Y., Lyu, M. R. T., & Cai, H. (2013). Toward fine-grained, unsupervised, scalable performance diagnosis for production cloud computing systems. *IEEE Transactions on Parallel and Distributed Systems*, *24*(6), 1245–1255.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

Nedelkoski, S., Bogatinovski, J., Acker, A., Cardoso, J., & Kao, O. (2020). Self-attentive classification-based anomaly detection in unstructured logs. In *2020 IEEE international conference on data mining* (pp. 1196–1201). IEEE.

Oliner, A., & Stearley, J. (2007). What supercomputers say: A study of five system logs. In *37th annual IEEE/IFIP international conference on dependable systems and networks* (pp. 575–584). IEEE.

Otomo, K., Kobayashi, S., Fukuda, K., & Esaki, H. (2019). Latent variable based anomaly detection in network system logs. *IEICE Transactions on Information and Systems*, *102*(9), 1644–1652.

Ott, H., Bogatinovski, J., Acker, A., Nedelkoski, S., & Kao, O. (2021). Robust and transferable anomaly detection in log data using pre-trained language models. In *2021 IEEE/ACM international workshop on cloud intelligence (CloudIntelligence)* (pp. 19–24). IEEE.

Patil, A., Wadekar, A., Gupta, T., Vijan, R., & Kazi, F. (2019). Explainable LSTM model for anomaly detection in HDFS log file using layerwise relevance propagation. In *2019 IEEE bombay section signature conference* (pp. 1–6). IEEE.

Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing* (pp. 1532–1543).

Preuveneers, D., Rimmer, V., Tsingenopoulos, I., Spooren, J., Joosen, W., & Ilie-Zudor, E. (2018). Chained anomaly detection models for federated learning: An intrusion detection case study. *Applied Sciences*, *8*(12), 2663.

Qian, Y., Ying, S., & Wang, B. (2020). Anomaly detection in distributed systems via variational autoencoders. In *2020 IEEE international conference on systems, man, and cybernetics* (pp. 2822–2829). IEEE.

Sarker, I. H. (2021a). Deep cybersecurity: A comprehensive overview from neural network and deep learning perspective. *SN Computer Science*, *2*(3), 1–16.

Sarker, I. H. (2021b). Deep learning: A comprehensive overview on techniques, taxonomy, applications and research directions. *SN Computer Science*, *2*(6), 1–20.

Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., & Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation*, *13*(7), 1443–1471.

Studiawan, H., & Sohel, F. (2021). Anomaly detection in a forensic timeline with deep autoencoders. *Journal of Information Security and Applications*, *63*, Article 103002.

Studiawan, H., Sohel, F., & Payne, C. (2020). Anomaly detection in operating system logs with deep learning-based sentiment analysis. *IEEE Transactions on Dependable and Secure Computing*, *18*(5), 2136–2148.

Sun, P., Yuepeng, E., Li, T., Wu, Y., Ge, J., You, J., et al. (2020). Context-aware learning for anomaly detection with imbalanced log data. In *2020 IEEE 22nd international conference on high performance computing and communications; IEEE 18th international conference on smart city; IEEE 6th international conference on data science and systems* (pp. 449–456). IEEE.

Sundqvist, T., Bhuyan, M. H., Forsman, J., & Elmroth, E. (2020). Boosted ensemble learning for anomaly detection in 5G RAN. In *IFIP international conference on artificial intelligence applications and innovations* (pp. 15–30). Springer.

Suriadi, S., Andrews, R., ter Hofstede, A. H., & Wynn, M. T. (2017). Event log imperfection patterns for process mining: Towards a systematic approach to cleaning event logs. *Information Systems*, *64*, 132–150.

Syngal, S., Verma, S., Karthik, K., Katyal, Y., & Ghosh, S. (2021). Server-language processing: A semi-supervised approach to server failure detection. In *2021 2nd international conference on computing, networks and internet of things* (pp. 1–7).

Tax, D. M., & Duin, R. P. (2004). Support vector data description. *Machine Learning*, *54*(1), 45–66.

Wadekar, A., Gupta, T., Vijan, R., & Kazi, F. (2019). Hybrid CAE-VAE for unsupervised anomaly detection in log file systems. In *2019 10th international conference on computing, communication and networking technologies* (pp. 1–7). IEEE.

Wan, Y., Liu, Y., Wang, D., & Wen, Y. (2021). GLAD-PAW: Graph-based log anomaly detection by position aware weighted graph attention network. In *Pacific-asia conference on knowledge discovery and data mining* (pp. 66–77). Springer.

Wang, Z., Chen, Z., Ni, J., Liu, H., Chen, H., & Tang, J. (2021). Multi-scale one-class recurrent neural networks for discrete event sequence anomaly detection. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining* (pp. 3726–3734).

Wang, Z., Tian, J., Fang, H., Chen, L., & Qin, J. (2022). LightLog: A lightweight temporal convolutional network for log anomaly detection on the edge. *Computer Networks*, *203*, Article 108616.

Wang, M., Xu, L., & Guo, L. (2018). Anomaly detection of system logs based on natural language processing and deep learning. In *2018 4th international conference on frontiers of signal processing* (pp. 140–144). IEEE.

Wang, Q., Zhang, X., Wang, X., & Cao, Z. (2021). Log sequence anomaly detection method based on contrastive adversarial training and dual feature extraction. *Entropy*, *24*(1), 69.

Wibisono, S. R., & Kistijantoro, A. I. (2019). Log anomaly detection using adaptive universal transformer. In *2019 international conference of advanced informatics: concepts, theory and applications* (pp. 1–6). IEEE.

Wittkopp, T., Acker, A., Nedelkoski, S., Bogatinovski, J., Scheinert, D., Fan, W., et al. (2021). A2log: Attentive augmented log anomaly detection. arXiv preprint arXiv:2109.09537.

Xi, L., Xin, Y., Luo, S., Shang, Y., & Tang, Q. (2021). Anomaly detection mechanism based on hierarchical weights through large-scale log data. In *2021 international conference on computer communication and artificial intelligence* (pp. 106–115). IEEE.

Xia, B., Bai, Y., Yin, J., Li, Y., & Xu, J. (2021). LogGAN: A log-level generative adversarial network for anomaly detection using permutation event modeling. *Information Systems Frontiers*, *23*(2), 285–298.

Xiao, C., Huang, J., & Wu, W. (2019). Detecting anomalies in cluster system using hybrid deep learning model. In *International symposium on parallel architectures, algorithms and programming* (pp. 393–404). Springer.

Xie, Y., Ji, L., & Cheng, X. (2020). An attention-based GRU network for anomaly detection from system logs. *IEICE Transactions on Information and Systems*, *103*(8), 1916–1919.

Xu, W., Huang, L., Fox, A., Patterson, D., & Jordan, M. (2009a). Largescale system problem detection by mining console logs. In *Proceedings of SOSP'09*. Citeseer.

Xu, W., Huang, L., Fox, A., Patterson, D., & Jordan, M. I. (2009b). Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd symposium on operating systems principles* (pp. 117–132).

Yadav, R. B., Kumar, P. S., & Dhavale, S. V. (2020). A survey on log anomaly detection using deep learning. In *2020 8th international conference on reliability, infocom technologies and optimization (trends and future directions)* (pp. 1215–1220). IEEE.

Yang, T., & Agrawal, V. (2016). Log file anomaly detection. *2016*, CS224d Fall. Online: https://cs224d.stanford.edu/reports/YangAgrawal.pdf.

Yang, L., Chen, J., Wang, Z., Wang, W., Jiang, J., Dong, X., et al. (2021). Semi-supervised log-based anomaly detection via probabilistic label estimation. In *2021 IEEE/ACM 43rd international conference on software engineering* (pp. 1448–1460). IEEE.

Yang, R., Qu, D., Gao, Y., Qian, Y., & Tang, Y. (2019). NLSALog: An anomaly detection framework for log sequence in security management. *IEEE Access*, *7*, 181152–181164.

Yen, S., Moh, M., & Moh, T. S. (2019). Causalconvlstm: Semi-supervised log anomaly detection through sequence modeling. In *2019 18th IEEE international conference on machine learning and applications* (pp. 1334–1341). IEEE.

Yin, K., Yan, M., Xu, L., Xu, Z., Li, Z., Yang, D., et al. (2020). Improving log-based anomaly detection with component-aware analysis. In *2020 IEEE international conference on software maintenance and evolution* (pp. 667–671). IEEE.

Yu, D., Hou, X., Li, C., Lv, Q., Wang, Y., & Li, N. (2021). Anomaly detection in unstructured logs using attention-based Bi-LSTM network. In *2021 7th IEEE international conference on network intelligence and digital content* (pp. 403–407). IEEE.

Zhang, D., Dai, D., Han, R., & Zheng, M. (2021). SentiLog: Anomaly detecting on parallel file systems via log-based sentiment analysis. In *Proceedings of the 13th ACM workshop on hot topics in storage and file systems* (pp. 86–93).

Zhang, L., Li, W., Zhang, Z., Lu, Q., Hou, C., Hu, P., et al. (2021). LogAttn: Unsupervised log anomaly detection with an AutoEncoder based attention mechanism. In *International conference on knowledge science, engineering and management* (pp. 222–235). Springer.

Zhang, Z., & Sabuncu, M. (2018). Generalized cross entropy loss for training deep neural networks with noisy labels. *Advances in Neural Information Processing Systems*, *31*.

Zhang, C., Wang, X., Zhang, H., Zhang, H., & Han, P. (2021). Log sequence anomaly detection based on local information extraction and globally sparse transformer model. *IEEE Transactions on Network and Service Management*, *18*(4), 4119–4133.

Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., et al. (2019). Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering* (pp. 807–817).

Zhao, Z., Niu, W., Zhang, X., Zhang, R., Yu, Z., & Huang, C. (2021). Trine: Syslog anomaly detection with three transformer encoders in one generative adversarial network. *Applied Intelligence*, 1–10.

Zhao, N., Wang, H., Li, Z., Peng, X., Wang, G., Pan, Z., et al. (2021). An empirical investigation of practical log anomaly detection for online service systems. In *Proceedings of the 29th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering* (pp. 1404–1415).

Zheng, Z., Yu, L., Tang, W., Lan, Z., Gupta, R., Desai, N., et al. (2011). Co-analysis of RAS log and job log on Blue Gene/P. In *2011 IEEE international parallel & distributed processing symposium* (pp. 840–851). IEEE.

Zhou, P., Wang, Y., Li, Z., Wang, X., Tyson, G., & Xie, G. (2020). Logsayer: Log pattern-driven cloud component anomaly diagnosis with machine learning. In *2020 IEEE/ACM 28th international symposium on quality of service* (pp. 1–10). IEEE.

Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z., et al. (2019). Tools and benchmarks for automated log parsing. In *2019 IEEE/ACM 41st international conference on software engineering: software engineering in practice* (pp. 121–130). IEEE.

Zhu, B., Li, J., Gu, R., & Wang, L. (2020). An approach to cloud platform log anomaly detection based on natural language processing and LSTM. In *2020 3rd international conference on algorithms, computing and artificial intelligence* (pp. 1–7).